```
%pip install pandas numpy scikit-learn matplotlib seaborn xgboost ope
```

```
import pandas as pd
import numpy as np
import re
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import spearmanr
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import LeaveOneOut, StratifiedKFold
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingCl
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix,
import warnings
warnings.filterwarnings('ignore')
np.random.seed(42)

# Set display options
pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
pd.set_option('display.max_colwidth', 50)

print("✓ Libraries imported successfully!")
```

```
✓ Libraries imported successfully!
```

Motor Balance Analysis: Preliminary Welding Exposure Stages vs Hoehn & Yahr Stages

Research Objective

Primary Objective: To develop preliminary welding exposure stages based on years of welding experience and to examine how these exposure stages relate to clinical balance outcomes (BBS, Mini-BESTest, FES), compared with Hoehn & Yahr stages in

individuals with Parkinson's disease.

Secondary Aims:

1. To compare balance performance using **symptom/clinical data** (BBS, MiniBEST, FES, ABC) across PD and Welders
2. To predict from **common data** present in both groups (no welding years); symptoms over years
3. Control data is **not used** in the analysis; only PD and Welders

Important Note: This is a **pilot/exploratory study** with a small sample size. The exposure staging system is **preliminary and not validated**. We examine patterns and relationships descriptively, not claiming equivalence or validation of a staging system.

Research Approach

1. **Develop Preliminary Welding Exposure Stages** (W1-W5) using simple, clinically-readable cutpoints:
   - **W1 (Very low exposure)**: 0 to <10 years
   - **W2 (Low-moderate)**: 10 to <20 years
   - **W3 (Moderate-high)**: 20 to <30 years
   - **W4 (High)**: 30 to <40 years
   - **W5 (Very high)**: ≥40 years
   - Additional exposure factors captured for analysis: noise, vibration, fume, hours, PPE
2. **Examine Relationships** (exploratory, nonparametric):
   - Spearman correlation between exposure stage and balance outcomes (BBS, MiniBEST, FES) in welders
   - Spearman correlation between H&Y stage and balance outcomes in PD
   - Descriptive comparison of balance patterns across stages (medians/means, visualizations)
   - Group comparisons: PD vs welders (symptom-based)
3. **Exploratory Modeling** (given small sample size):
   - Motor balance severity classification
   - Fall risk prediction

- Using Logistic Regression and Random Forest (marked as exploratory)

Study Populations

- **PD Patients**: Classified by Hoehn & Yahr stages (1-5) - disease progression stages
- **Welders**: Classified by preliminary Welding Exposure Stages (W1-W5) - exposure-based staging
- **Controls**: Not used in this analysis (PD and Welders only)

1. Data Loading

```
# Load data file
# For Colab: Upload PD_WELDERS RAW Long Data.xlsx using the file brows
# For local: Place the file in the same directory as this notebook
xlsx_path = "PD_WELDERS RAW Long Data.xlsx"

# Load data
xls = pd.ExcelFile(xlsx_path)
pd_df = pd.read_excel(xls, "PD")
weld_df = pd.read_excel(xls, "WD")
# Control data is not used in this analysis; only PD and Welders are

# Clean column names
pd_df.columns = [c.strip() for c in pd_df.columns]
weld_df.columns = [c.strip() for c in weld_df.columns]

# Remove empty rows
pd_df = pd_df[~pd_df["Participant ID"].isna()].copy()
weld_df = weld_df[~weld_df["Participant's Name"].isna()].copy() if "Pa

print(f"\n{'='*80}")
print(f"Data Loaded Successfully!")
print(f"{'='*80}")
print(f"Data: PD={len(pd_df)}, Welders={len(weld_df)} (control not use
print(f"\nPD columns with 'Stage': {[c for c in pd_df.columns if 'Stag
print(f"\nWelder columns with balance scores: {[c for c in weld_df.co
# Verify Excel data (updated file)
print("--- Data verification (Excel) ---")
print("Sheet names:", xls.sheet_names)
print("PD shape:", pd_df.shape, "| WD shape:", weld_df.shape)
```

```
print("PD columns:", list(pd_df.columns))
print("WD columns:", list(weld_df.columns))
```

```
==================================================================
Data Loaded Successfully!
==================================================================
Data: PD=14, Welders=16 (control not used)

PD columns with 'Stage': ['Current Stage of PD (Hoehn & Yahr)']

Welder columns with balance scores: ['BBS', 'MINI-BEST SCORE', 'FES']
--- Data verification (Excel) ---
Sheet names: ['PD', 'WD', 'C']
PD shape: (14, 74) | WD shape: (16, 79)
PD columns: ['Participant ID', 'Age (in years)', 'Gender', 'Marital Sta
WD columns: ['Participant’s Name', 'Group', 'Age', 'Gender', 'Marital S
```

2. Feature Extraction and Engineering

```
# ==================================================================
# CLINICAL SCALE ITEMS
# ==================================================================
BBS_ITEMS = ["Sitting to Standing", "Standing Unsupported", "Sitting U

MINIBEST_ITEMS = ["Sit to Stand (Anticipatory Postural Adjustments)",

ABC_ITEMS = ["How confident are you in Taking a bath or shower withou

# ==================================================================
# HELPER FUNCTIONS
# ==================================================================
def calc_score(df, items):
    """Calculate total score from item columns"""
    available = [c for c in items if c in df.columns]
    return df[available].apply(pd.to_numeric, errors="coerce").sum(ax

def parse_hoehn_yahr_stage(val):
    """Parse Hoehn & Yahr stage (1-5) from string"""
    if pd.isna(val): return 0
    s = str(val).upper().strip()
    if "V" in s or "5" in s: return 5
```

```
    if "IV" in s or "4" in s: return 4
    if "III" in s or "3" in s: return 3
    if "II" in s or "2" in s: return 2
    if "I" in s or "1" in s: return 1
    return 0

def map_fall(val, is_pd=False):
    """Map fall history to binary (0/1)"""
    if pd.isna(val): return 0
    s = str(val).strip().lower()
    if is_pd:
        return 0 if s in ["no", "0", "none", ""] else 1
    return 1 if s in ["yes", "y", "1"] else 0

def score_exposure_level(val):
    """Score exposure frequency: Never=0, Occasional=1, Frequent=2, C
    if pd.isna(val): return 0
    s = str(val).strip().lower()
    if "never" in s: return 0
    if "occasional" in s: return 1
    if "frequent" in s: return 2
    if "constant" in s: return 3
    return 1  # default

def score_ppe_level(val):
    """Score PPE use: Never=0, Sometimes=1, Always=2"""
    if pd.isna(val): return 0
    s = str(val).strip().lower()
    if "never" in s: return 0
    if "sometimes" in s: return 1
    if "always" in s: return 2
    return 0  # default

def score_welding_type(val):
    """Score welding type: Arc=1, MIG=2, Others=1.5"""
    if pd.isna(val): return 1
    s = str(val).strip().lower()
    if "arc" in s: return 1.0
    if "mig" in s: return 2.0
    return 1.5  # other types

def assign_welding_exposure_stage(years):
    """
    Assign preliminary welding exposure stage (W1-W5) based on years
    Simple cutpoint approach for clinical readability.
```

```
        W1 (Very low exposure): 0–<10 years
        W2 (Low–moderate): 10–<20 years
        W3 (Moderate–high): 20–<30 years
        W4 (High): 30–<40 years
        W5 (Very high): ≥40 years
        """
        if pd.isna(years) or years <= 0:
            return 0
        if years < 10:
            return 1  # W1
        elif years < 20:
            return 2  # W2
        elif years < 30:
            return 3  # W3
        elif years < 40:
            return 4  # W4
        else:
            return 5  # W5

    def assign_motor_balance_severity(bbs, minibest, fes):
        """Unified motor balance severity classification (1–5)"""
        if pd.isna(bbs): bbs = 0
        if pd.isna(minibest): minibest = 0
        if pd.isna(fes): fes = 0

        if bbs < 25 or (bbs < 30 and minibest < 8):
            return 5  # Very Severe
        elif bbs < 38:
            return 4  # Severe
        elif bbs < 45 or (bbs < 50 and minibest < 19):
            return 3  # Moderate
        elif bbs < 52 or (bbs < 54 and fes < 85):
            return 2  # Mild
        else:
            return 1  # Normal

    def extract_pd(df):
        """Extract PD patient data with Hoehn & Yahr stages"""
        r = pd.DataFrame()
        n = len(df)
        r['Group'] = ['PD'] * n
        r['Age'] = pd.to_numeric(df["Age (in years)"], errors="coerce").va
        r['BBS'] = calc_score(df, BBS_ITEMS).values
        r['MiniBEST'] = calc_score(df, MINIBEST_ITEMS).values
```

```
    r['ABC'] = calc_score(df, ABC_ITEMS).values
    r['FES'] = pd.to_numeric(df.get("FES", pd.Series([0]*n)), errors='
    r['Hoehn_Yahr_Stage'] = df["Current Stage of PD (Hoehn & Yahr)"].a
    r['Stage'] = r['Hoehn_Yahr_Stage'].values
    r['Motor_Balance_Severity'] = [assign_motor_balance_severity(r['BI
    r['Stage_num'] = r['Motor_Balance_Severity'].values
    r['Severity'] = r['Motor_Balance_Severity'].values
    r['Welding_Stage'] = 0
    r['fall_flag'] = df["Number of Falls in the Last 6 Months"].apply
    r['exposure_years'] = pd.to_numeric(df["Disease Duration (years/m
    return r

def extract_welder(df):
    """Extract welder data with motor imbalance severity levels"""
    r = pd.DataFrame()
    n = len(df)
    r['Group'] = ['Welder'] * n
    r['Age'] = pd.to_numeric(df["Age"], errors="coerce").values
    if "BBS" in df.columns:
        r['BBS'] = pd.to_numeric(df["BBS"], errors="coerce").fillna(0
    else:
        r['BBS'] = calc_score(df, BBS_ITEMS).values
    if "MINI-BEST SCORE" in df.columns:
        r['MiniBEST'] = pd.to_numeric(df["MINI-BEST SCORE"], errors="
    else:
        r['MiniBEST'] = calc_score(df, MINIBEST_ITEMS).values
    if "FES" in df.columns:
        r['FES'] = pd.to_numeric(df["FES"], errors="coerce").fillna(0
    else:
        r['FES'] = pd.Series([0]*n).values
    r['ABC'] = calc_score(df, ABC_ITEMS).values
    r['Motor_Balance_Severity'] = [assign_motor_balance_severity(r['BI
    r['Welding_Stage'] = pd.to_numeric(df["Total Years in Welding"], 
    r['Stage_num'] = r['Motor_Balance_Severity'].values
    r['Severity'] = r['Motor_Balance_Severity'].values
    r['Hoehn_Yahr_Stage'] = 0
    r['Stage'] = 0
    r['fall_flag'] = df["History of Fall"].apply(lambda x: map_fall(x
    r['exposure_years'] = pd.to_numeric(df["Total Years in Welding"],
    return r

def extract_control(df):
    """Extract control data"""
    r = pd.DataFrame()
    n = len(df)
```

```
        r['Group'] = ['Control'] * n
        age_col = None
        for col in df.columns:
            if 'age' in col.lower() or 'Age' in col:
                age_col = col
                break
        r['Age'] = pd.to_numeric(df[age_col] if age_col else pd.Series([0
        r['BBS'] = calc_score(df, BBS_ITEMS).values
        r['MiniBEST'] = calc_score(df, MINIBEST_ITEMS).values
        r['ABC'] = calc_score(df, ABC_ITEMS).values
        r['FES'] = pd.to_numeric(df.get("FES", pd.Series([0]*n)), errors='
        r['Hoehn_Yahr_Stage'] = 0
        r['Stage'] = 0
        r['Stage_num'] = 0
        r['Severity'] = 0
        r['Motor_Balance_Severity'] = [assign_motor_balance_severity(r['BI
        r['Welding_Stage'] = 0
        r['fall_flag'] = 0
        r['exposure_years'] = 0
        return r

    # Extract data: PD and Welders only (no control data used in analysis
    data = pd.concat([
        extract_pd(pd_df),
        extract_welder(weld_df)
    ], ignore_index=True)

    # Remove rows with missing critical data
    data = data.dropna(subset=["Age", "BBS"])
    print(f"Total samples: {len(data)} (PD + Welders only; control data n
    print(f"\nUnified Motor Balance Severity distribution:")
    print(data['Motor_Balance_Severity'].value_counts().sort_index())
    print(f"\nPD - Hoehn & Yahr Stage distribution:")
    print(data[data['Group'] == 'PD']['Hoehn_Yahr_Stage'].value_counts().
    print(f"\nMotor Balance Severity by Group:")
    print(pd.crosstab(data['Group'], data['Motor_Balance_Severity']))
```

```
Total samples: 30 (PD + Welders only; control data not used)

Unified Motor Balance Severity distribution:
Motor_Balance_Severity
1    4
2    7
3    7
4    6
5    6
```

```
Name: count, dtype: int64

PD - Hoehn & Yahr Stage distribution:
Hoehn_Yahr_Stage
1    2
2    3
3    6
5    3
Name: count, dtype: int64

Motor Balance Severity by Group:
Motor_Balance_Severity  1  2  3  4  5
Group
PD                      0  2  2  6  4
Welder                  4  5  5  0  2
```

```
# ======================================================================
# COMPREHENSIVE DATA EXTRACTION: PD PATIENTS
# ======================================================================
def extract_pd(df):
    """Extract PD patient data with all factors"""
    r = pd.DataFrame()
    n = len(df)

    # Group identifier
    r['Group'] = ['PD'] * n

    # Demographics & Health Factors
    r['Age'] = pd.to_numeric(df["Age (in years)"], errors="coerce").va

    # Clinical Motor & Balance Factors
    r['BBS'] = calc_score(df, BBS_ITEMS).values
    r['MiniBEST'] = calc_score(df, MINIBEST_ITEMS).values
    r['ABC'] = calc_score(df, ABC_ITEMS).values
    r['FES'] = pd.to_numeric(df.get("FES", pd.Series([0]*n)), errors='

    # Stage Classification (Hoehn & Yahr)
    r['Hoehn_Yahr_Stage'] = df["Current Stage of PD (Hoehn & Yahr)"].a

    # Motor Balance Severity
    r['Motor_Balance_Severity'] = [assign_motor_balance_severity(r['BI

    # Fall History
    r['fall_flag'] = df["Number of Falls in the Last 6 Months"].apply

    # Disease Duration (exposure_years for PD = disease duration)
```

```
    r['exposure_years'] = pd.to_numeric(df["Disease Duration (years/m

    # Health Factors (if available)
    r['MMSE'] = pd.to_numeric(df.get("MMSE (/30)", pd.Series([0]*n)),
    r['has_comorbidities'] = (~df.get("Comorbidities", pd.Series([""]

    # Set welder-specific fields to 0
    r['Welding_Stage'] = 0
    r['Exposure_Index'] = 0
    r['Noise_Exposure'] = 0
    r['Vibration_Exposure'] = 0
    r['Fume_Exposure'] = 0
    r['Work_Hours_Day'] = 0
    r['PPE_Score'] = 0
    r['Welding_Type_Score'] = 0

    return r

# ==================================================================
# COMPREHENSIVE DATA EXTRACTION: WELDERS
# ==================================================================
def extract_welder(df):
    """Extract welder data with ALL exposure factors + clinical data"
    r = pd.DataFrame()
    n = len(df)

    # Group identifier
    r['Group'] = ['Welder'] * n

    # Demographics & Health Factors
    r['Age'] = pd.to_numeric(df["Age"], errors="coerce").values

    # Clinical Motor & Balance Factors
    if "BBS" in df.columns:
        r['BBS'] = pd.to_numeric(df["BBS"], errors="coerce").fillna(0
    else:
        r['BBS'] = calc_score(df, BBS_ITEMS).values

    if "MINI-BEST SCORE" in df.columns:
        r['MiniBEST'] = pd.to_numeric(df["MINI-BEST SCORE"], errors="
    else:
        r['MiniBEST'] = calc_score(df, MINIBEST_ITEMS).values

    if "FES" in df.columns:
        r['FES'] = pd.to_numeric(df["FES"], errors="coerce").fillna(0
```

```
else:
    r['FES'] = pd.Series([0]*n).values

r['ABC'] = calc_score(df, ABC_ITEMS).values

# Motor Balance Severity
r['Motor_Balance_Severity'] = [assign_motor_balance_severity(r['BI

# Fall History
r['fall_flag'] = df["History of Fall"].apply(lambda x: map_fall(x

# ==================================================================
# EXPOSURE-RELATED FACTORS (Bucket 1)
# ==================================================================
# Total years in welding
r['exposure_years'] = pd.to_numeric(df["Total Years in Welding"],

# Intensity factors
r['Noise_Exposure'] = df.get("Occupational Exposure [Noise Exposu
r['Vibration_Exposure'] = df.get("Occupational Exposure [Vibratio
r['Fume_Exposure'] = df.get("Occupational Exposure [Exposure to F

# Work hours per day
r['Work_Hours_Day'] = pd.to_numeric(df.get("Work Hours per Day", p

# Type of welding
r['Welding_Type_Score'] = df.get("Type of Welding", pd.Series([""

# PPE use (protection level - inverse: higher PPE = lower effecti
hearing_ppe = df.get("Use of Personal Protective Equipment (PPE)
resp_ppe = df.get("Use of Personal Protective Equipment (PPE) [Res
eye_ppe = df.get("Use of Personal Protective Equipment (PPE) [Eye
r['PPE_Score'] = (hearing_ppe + resp_ppe + eye_ppe) / 3.0  # Avera

# ==================================================================
# CUMULATIVE EXPOSURE INDEX
# ==================================================================
# exposure_index = years × hours/day × (noise + vibration + fume s
intensity_sum = r['Noise_Exposure'] + r['Vibration_Exposure'] + r
ppe_protection = r['PPE_Score'] / 2.0  # Normalize to 0-1
r['Exposure_Index'] = (r['exposure_years'] *
                       (r['Work_Hours_Day'] / 8.0) *  # Normalize t
                       (intensity_sum / 9.0) *  # Normalize to max
                       r['Welding_Type_Score'] *
                       (1.0 - ppe_protection * 0.5))  # PPE reduces
```

```
    # Health Factors (Bucket 3)
    r['has_comorbidities'] = (~df.get("Do you have any of the followi
    r['Exercise_Frequency'] = df.get("Exercise Frequency", pd.Series(

    # Set PD-specific fields to 0
    r['Hoehn_Yahr_Stage'] = 0
    r['MMSE'] = 0

    # Assign preliminary Welding Exposure Stage (W1-W5) based on years
    r['Welding_Stage'] = r['exposure_years'].apply(assign_welding_expo

    return r

# ======================================================================
# EXTRACT DATA FROM ALL GROUPS
# ======================================================================
print("Extracting data from all groups...")
pd_data = extract_pd(pd_df)
welder_data = extract_welder(weld_df)

# Controls (set aside, not used in main comparison)
ctrl_data = pd.DataFrame()  # Will extract if needed for reference

# Combine PD and Welders for main analysis
data = pd.concat([pd_data, welder_data], ignore_index=True)

# Remove rows with missing critical data
data = data.dropna(subset=["Age", "BBS"])

print(f"\n{'='*80}")
print(f"ETL Pipeline Complete!")
print(f"{'='*80}")
print(f"Total samples: {len(data)}")
print(f"  - PD: {len(pd_data)}")
print(f"  - Welders: {len(welder_data)}")
print(f"\nPD - Hoehn & Yahr Stage distribution:")
print(data[data['Group'] == 'PD']['Hoehn_Yahr_Stage'].value_counts().s
print(f"\nWelder - Exposure Index stats:")
print(welder_data['Exposure_Index'].describe())
print(f"\nMotor Balance Severity by Group:")
print(pd.crosstab(data['Group'], data['Motor_Balance_Severity']))
```

```
Extracting data from all groups...
```

```
==================================================================================
ETL Pipeline Complete!
==================================================================================
Total samples: 30
  - PD: 14
  - Welders: 16

PD - Hoehn & Yahr Stage distribution:
Hoehn_Yahr_Stage
1    2
2    3
3    6
5    3
Name: count, dtype: int64

Welder - Exposure Index stats:
count    16.000000
mean     15.735822
std      13.794231
min       1.777778
25%       5.925926
50%      10.092593
75%      22.638889
max      44.444444
Name: Exposure_Index, dtype: float64

Motor Balance Severity by Group:
Motor_Balance_Severity  1  2  3  4  5
Group
PD                      0  2  2  6  4
Welder                  4  5  5  0  2
```

1.3 Preliminary Welding Exposure Stage Assignment

Simple cutpoint-based staging for clinical readability:

- **W1 (Very low exposure)**: 0-<10 years
- **W2 (Low-moderate)**: 10-<20 years
- **W3 (Moderate-high)**: 20-<30 years
- **W4 (High)**: 30-<40 years
- **W5 (Very high)**: ≥40 years

Note: This is a preliminary staging system based on years of exposure. Additional exposure factors (noise, vibration, fume, PPE) are captured for analysis but not used for primary staging to maintain simplicity and clinical interpretability.

```
# ======================================================================
# PRELIMINARY WELDING EXPOSURE STAGE ASSIGNMENT
# ======================================================================
# Simple cutpoint-based staging (already assigned in extract_welder fun
# Display stage distribution and descriptive statistics

welder_subset = data[data['Group'] == 'Welder'].copy()

if len(welder_subset) > 0:
    print(f"{'='*80}")
    print("Preliminary Welding Exposure Stage Assignment")
    print(f"{'='*80}")
    print("\nStage Definitions:")
    print("  W1 (Very low exposure): 0-<10 years")
    print("  W2 (Low-moderate): 10-<20 years")
    print("  W3 (Moderate-high): 20-<30 years")
    print("  W4 (High): 30-<40 years")
    print("  W5 (Very high): ≥40 years")

    print(f"\n{'='*80}")
    print("Welding Stage Distribution by Years of Exposure:")
    print(f"{'='*80}")
    stage_dist = welder_subset.groupby('Welding_Stage').agg({
        'exposure_years': ['count', 'mean', 'std', 'min', 'max'],
        'BBS': ['mean', 'std'],
        'MiniBEST': ['mean', 'std'],
        'FES': ['mean', 'std'],
        'Exposure_Index': ['mean', 'std']
```

```
        Exposure_Index': ['mean', 'std'],
    }).round(2)
    print(stage_dist)

    print(f"\n{'='*80}")
    print("Stage-by-Stage Summary:")
    print(f"{'='*80}")
    for stage in sorted(welder_subset['Welding_Stage'].unique()):
        if stage > 0:
            stage_data = welder_subset[welder_subset['Welding_Stage'] =
            if len(stage_data) > 0:
                print(f"\nW-Stage {int(stage)} (n={len(stage_data)}):")
                print(f"  Years: {stage_data['exposure_years'].mean():.
                print(f"  BBS: {stage_data['BBS'].mean():.1f}±{stage_da
                print(f"  MiniBEST: {stage_data['MiniBEST'].mean():.1f}
                print(f"  FES: {stage_data['FES'].mean():.1f}±{stage_da
                print(f"  Exposure Index: {stage_data['Exposure_Index']
else:
    print("No welder data available for stage assignment")
```

```
  W5 (Very high): ≥40 years

================================================================================
Welding Stage Distribution by Years of Exposure:
================================================================================
              exposure_years                          BBS        MiniB
                       count   mean   std min max   mean    std     m
Welding_Stage
1                          4   8.50  0.58   8   9  42.25  13.62    20
2                          4  13.25  3.20  10  16  52.00   2.94    21
3                          4  22.25  2.63  20  25  52.00   3.37    24
4                          3  33.00  1.73  32  35  50.33   4.04    17
5                          1  40.00   NaN  40  40  22.00    NaN     1

                       FES        Exposure_Index
                std   mean    std           mean    std
Welding_Stage
1              5.35  31.00  31.60           5.56   4.79
2              4.57  14.50   3.79           5.53   1.99
3              3.37  18.25   5.74          21.84  15.81
4              8.08  60.00   0.00          29.83  10.68
5               NaN  80.00    NaN          30.56    NaN

================================================================================
Stage-by-Stage Summary:
================================================================================

W-Stage 1 (n=4):
```

```
    Years: 8.5±0.6 (range: 8-9)
    BBS: 42.2±13.6
    MiniBEST: 20.0±5.4
    FES: 31.0±31.6
    Exposure Index: 5.6±4.8

  W-Stage 2 (n=4):
    Years: 13.2±3.2 (range: 10-16)
    BBS: 52.0±2.9
    MiniBEST: 21.8±4.6
    FES: 14.5±3.8
    Exposure Index: 5.5±2.0

  W-Stage 3 (n=4):
    Years: 22.2±2.6 (range: 20-25)
    BBS: 52.0±3.4
    MiniBEST: 24.0±3.4
    FES: 18.2±5.7
    Exposure Index: 21.8±15.8

  W-Stage 4 (n=3):
    Years: 33.0±1.7 (range: 32-35)
    BBS: 50.3±4.0
    MiniBEST: 17.7±8.1
    FES: 60.0±0.0
    Exposure Index: 29.8±10.7

  W-Stage 5 (n=1):
    Years: 40.0±nan (range: 40-40)
    BBS: 22.0±nan
    MiniBEST: 1.0±nan
```

2. Exploratory Analysis: Correlations and Relationships

Examine relationships between stages and balance outcomes using nonparametric correlations (Spearman's rho)

```
# ====================================================================
# EXPLORATORY CORRELATION ANALYSIS
# ====================================================================
# PD and Welders only (no control data)
analysis_data = data[data['Group'].isin(['PD', 'Welder'])].copy()

print("="*80)
print("EXPLORATORY CORRELATION ANALYSIS")
```

```
print("="*80)

# PD: Correlation between H&Y stage and balance outcomes
pd_data = analysis_data[analysis_data['Group'] == 'PD'].copy()
if len(pd_data) > 0:
    print("\n1. PD Patients: Hoehn & Yahr Stage vs Balance Outcomes")
    print("-" * 80)
    for outcome in ['BBS', 'MiniBEST', 'FES']:
        valid = pd_data[[outcome, 'Hoehn_Yahr_Stage']].dropna()
        if len(valid) > 2:
            rho, p = spearmanr(valid['Hoehn_Yahr_Stage'], valid[outcor
            print(f"   H&Y Stage vs {outcome:10s}: ρ = {rho:6.3f}, p =

# Welders: Correlation between Welding Stage and balance outcomes
welder_data = analysis_data[analysis_data['Group'] == 'Welder'].copy(
if len(welder_data) > 0:
    print("\n2. Welders: Welding Exposure Stage vs Balance Outcomes")
    print("-" * 80)
    for outcome in ['BBS', 'MiniBEST', 'FES']:
        valid = welder_data[[outcome, 'Welding_Stage']].dropna()
        if len(valid) > 2:
            rho, p = spearmanr(valid['Welding_Stage'], valid[outcome]
            print(f"   W-Stage vs {outcome:10s}: ρ = {rho:6.3f}, p = 

    print("\n3. Welders: Exposure Years vs Balance Outcomes")
    print("-" * 80)
    for outcome in ['BBS', 'MiniBEST', 'FES']:
        valid = welder_data[[outcome, 'exposure_years']].dropna()
        if len(valid) > 2:
            rho, p = spearmanr(valid['exposure_years'], valid[outcome
            print(f"   Years vs {outcome:10s}: ρ = {rho:6.3f}, p = {p

    print("\n4. Welders: Exposure Index vs Balance Outcomes")
    print("-" * 80)
    for outcome in ['BBS', 'MiniBEST', 'FES']:
        valid = welder_data[[outcome, 'Exposure_Index']].dropna()
        if len(valid) > 2:
            rho, p = spearmanr(valid['Exposure_Index'], valid[outcome
            print(f"   Exp Index vs {outcome:10s}: ρ = {rho:6.3f}, p =

print("\n" + "="*80)
print("Note: These correlations are exploratory given the small sampl
print("="*80)

# ==================================================================
```

```
# FEATURE ENGINEERING (for modeling)
# ==================================================================

# Normalized scores
analysis_data['bbs_norm'] = analysis_data['BBS'] / 56
analysis_data['minibest_norm'] = analysis_data['MiniBEST'] / 28
analysis_data['fes_norm'] = analysis_data['FES'] / 100
analysis_data['abc_norm'] = analysis_data['ABC'] / 100

# Deficit scores
analysis_data['bbs_deficit'] = 56 - analysis_data['BBS']
analysis_data['minibest_deficit'] = 28 - analysis_data['MiniBEST']
analysis_data['fes_deficit'] = 100 - analysis_data['FES']

# Composite measures
analysis_data['composite_balance'] = (analysis_data['bbs_norm'] + ana
analysis_data['balance_impaired'] = ((analysis_data['BBS'] < 45) | (a
analysis_data['high_fall_risk'] = (analysis_data['BBS'] < 45).astype(

# Interaction terms
analysis_data['age_bbs'] = analysis_data['Age'] * analysis_data['bbs_
analysis_data['age_fes'] = analysis_data['Age'] * analysis_data['fes_
analysis_data['age_minibest'] = analysis_data['Age'] * analysis_data[
analysis_data['bbs_minibest_ratio'] = analysis_data['BBS'] / (analysi
analysis_data['fes_bbs_diff'] = analysis_data['fes_norm'] - analysis_

# Exposure interactions
analysis_data['exposure_bbs'] = analysis_data['exposure_years'] * ana
analysis_data['exposure_fes'] = analysis_data['exposure_years'] * ana

# Stage interactions
# PD: Hoehn & Yahr stage interactions
analysis_data['hy_stage_bbs'] = analysis_data['Hoehn_Yahr_Stage'] * a
analysis_data['hy_stage_fes'] = analysis_data['Hoehn_Yahr_Stage'] * a
analysis_data['hy_stage_minibest'] = analysis_data['Hoehn_Yahr_Stage'

# Welders: Welding Stage interactions
analysis_data['weld_stage_bbs'] = analysis_data['Welding_Stage'] * an
analysis_data['weld_stage_fes'] = analysis_data['Welding_Stage'] * an
analysis_data['weld_stage_minibest'] = analysis_data['Welding_Stage']

# Unified severity interactions
analysis_data['severity_bbs'] = analysis_data['Motor_Balance_Severity
analysis_data['severity_fes'] = analysis_data['Motor_Balance_Severity
analysis_data['severity_minibest'] = analysis_data['Motor_Balance_Sev
```

```
# Exposure index interactions (welders only)
analysis_data['exposure_index_bbs'] = analysis_data['Exposure_Index']
analysis_data['exposure_index_fes'] = analysis_data['Exposure_Index']

print(f"Feature engineering complete!")
print(f"Total features created: {len([c for c in analysis_data.columns
print(f"\nAnalysis dataset: {len(analysis_data)} samples (PD: {len(ana
```

```
======================================================================
EXPLORATORY CORRELATION ANALYSIS
======================================================================

1. PD Patients: Hoehn & Yahr Stage vs Balance Outcomes
----------------------------------------------------------------------
   H&Y Stage vs BBS       : ρ = -0.553, p = 0.040 (n=14)
   H&Y Stage vs MiniBEST  : ρ = -0.446, p = 0.110 (n=14)
   H&Y Stage vs FES       : ρ =  0.564, p = 0.036 (n=14)

2. Welders: Welding Exposure Stage vs Balance Outcomes
----------------------------------------------------------------------
   W-Stage vs BBS       : ρ = -0.028, p = 0.917 (n=16)
   W-Stage vs MiniBEST  : ρ = -0.206, p = 0.445 (n=16)
   W-Stage vs FES       : ρ =  0.480, p = 0.060 (n=16)

3. Welders: Exposure Years vs Balance Outcomes
----------------------------------------------------------------------
   Years vs BBS       : ρ =  0.032, p = 0.908 (n=16)
   Years vs MiniBEST  : ρ = -0.136, p = 0.614 (n=16)
   Years vs FES       : ρ =  0.517, p = 0.040 (n=16)

4. Welders: Exposure Index vs Balance Outcomes
----------------------------------------------------------------------
   Exp Index vs BBS       : ρ = -0.095, p = 0.726 (n=16)
   Exp Index vs MiniBEST  : ρ = -0.214, p = 0.426 (n=16)
   Exp Index vs FES       : ρ =  0.465, p = 0.069 (n=16)

======================================================================
Note: These correlations are exploratory given the small sample size.
======================================================================
Feature engineering complete!
Total features created: 48

Analysis dataset: 30 samples (PD: 14, Welders: 16)
```

3. Feature Engineering

Create derived features for exploratory predictive modeling.

3. Stage Assignment & Feature Engineering

Primary Focus: Assign stages to both groups:

- **PD**: Hoehn & Yahr stages (1-5) from clinical assessment
- **Welders**: Welding Exposure stages (W-Stage I-V) from years of welding

```
# Stage assignment is already done in extraction functions
# PD: Hoehn_Yahr_Stage (1-5)
# Welders: Welding_Stage (1-5, W-Stage I-V)

data['high_fall_risk'] = (data['BBS'] < 45).astype(int)
data['bbs_norm'] = data['BBS'] / 56
data['minibest_norm'] = data['MiniBEST'] / 28
data['fes_norm'] = data['FES'] / 100
data['abc_norm'] = data['ABC'] / 100

data['bbs_deficit'] = 56 - data['BBS']
data['minibest_deficit'] = 28 - data['MiniBEST']
data['fes_deficit'] = 100 - data['FES']

data['composite_balance'] = (data['bbs_norm'] + data['minibest_norm']
data['balance_impaired'] = ((data['BBS'] < 45) | (data['MiniBEST'] < 

data['age_bbs'] = data['Age'] * data['bbs_norm']
data['age_fes'] = data['Age'] * data['fes_norm']
data['age_minibest'] = data['Age'] * data['minibest_norm']
data['bbs_minibest_ratio'] = data['BBS'] / (data['MiniBEST'] + 1)
data['fes_bbs_diff'] = data['fes_norm'] - data['bbs_norm']
data['exposure_bbs'] = data['exposure_years'] * data['bbs_norm']
data['exposure_fes'] = data['exposure_years'] * data['fes_norm']
# Stage interactions with balance scores
# PD: Hoehn & Yahr stage interactions
data['hy_stage_bbs'] = data['Hoehn_Yahr_Stage'] * data['bbs_norm']
data['hy_stage_fes'] = data['Hoehn_Yahr_Stage'] * data['fes_norm']
data['hy_stage_minibest'] = data['Hoehn_Yahr_Stage'] * data['minibest_
```

```
# Welders: Welding Exposure stage interactions
data['weld_stage_bbs'] = data['Welding_Stage'] * data['bbs_norm']
data['weld_stage_fes'] = data['Welding_Stage'] * data['fes_norm']
data['weld_stage_minibest'] = data['Welding_Stage'] * data['minibest_n

# Unified severity interactions
data['severity_bbs'] = data['Motor_Balance_Severity'] * data['bbs_norm
data['severity_fes'] = data['Motor_Balance_Severity'] * data['fes_norm
data['severity_minibest'] = data['Motor_Balance_Severity'] * data['min

print("Stage Distribution:")
print("\nPD - Hoehn & Yahr Stages:")
print(data[data['Group'] == 'PD']['Hoehn_Yahr_Stage'].value_counts().s
print("\nWelders - Welding Exposure Stages:")
print(data[data['Group'] == 'Welder']['Welding_Stage'].value_counts()
print("\nCross-tabulation:")
display(pd.crosstab(data['Group'], [data['Hoehn_Yahr_Stage'], data['We
```

```
Stage Distribution:

PD - Hoehn & Yahr Stages:
Hoehn_Yahr_Stage
1    2
2    3
3    6
5    3
Name: count, dtype: int64

Welders - Welding Exposure Stages:
Welding_Stage
1    4
2    4
3    4
4    3
5    1
Name: count, dtype: int64

Cross-tabulation:
```

Hoehn_Yahr_Stage	0					1	2	3	5	All
Welding_Stage	1	2	3	4	5	0	0	0	0	
Group										
PD	0	0	0	0	0	2	3	6	3	14
Welder	4	4	4	3	1	0	0	0	0	16

All	4	4	4	3	1	2	3	6	3	30

4. Descriptive Statistics by Group

```
summary_stats = data.groupby('Group')[['Age', 'exposure_years', 'BBS',
print("Table 1: Descriptive Statistics by Group")
display(summary_stats)
```

Table 1: Descriptive Statistics by Group

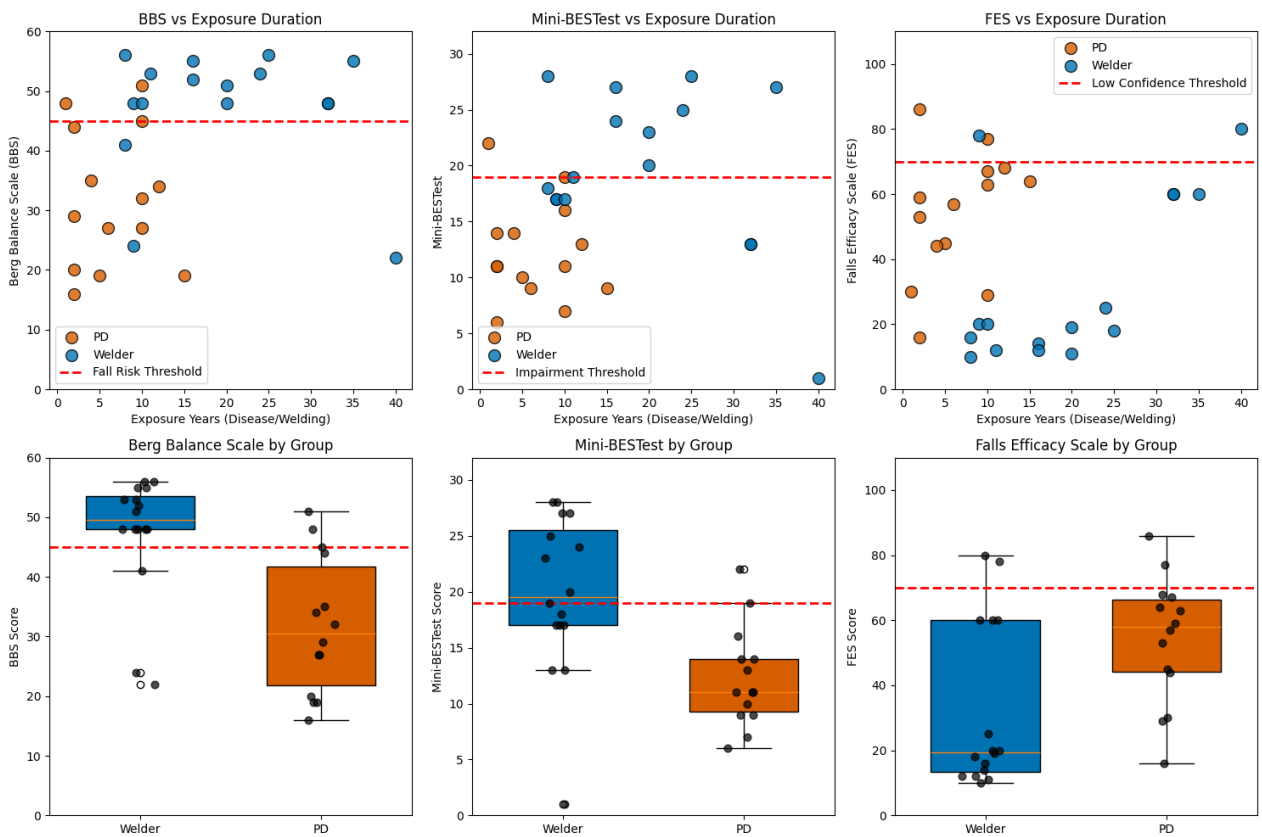
	Age				exposure_years				BBS				
	mean	std	min	max	mean	std	min	max	mean	std	min	max	r
Group													
PD	71.71	6.17	62	80	6.50	4.57	1	15	31.86	11.57	16	51	1
Welder	41.81	6.27	28	53	19.69	10.63	8	40	47.38	10.31	22	56	1

5. Motor Balance Comparison by Exposure Slabs

```
colors = {'PD': '#D55E00', 'Welder': '#0072B2'}

fig, axes = plt.subplots(2, 3, figsize=(15, 10))

ax1 = axes[0, 0]
for group in ['PD', 'Welder']:
    gd = data[data['Group'] == group]
    ax1.scatter(gd['exposure_years'], gd['BBS'], c=colors[group], s=1(
ax1.axhline(45, color='red', linestyle='--', lw=2, label='Fall Risk Tl
ax1.set_xlabel('Exposure Years (Disease/Welding)')
ax1.set_ylabel('Berg Balance Scale (BBS)')
ax1.set_title('BBS vs Exposure Duration')
ax1.legend()
ax1.set_ylim(0, 60)

ax2 = axes[0, 1]
for group in ['PD', 'Welder']:
    gd = data[data['Group'] == group]
```

```
        ax2.scatter(gd['exposure_years'], gd['MiniBEST'], c=colors[group],
    ax2.axhline(19, color='red', linestyle='--', lw=2, label='Impairment T
    ax2.set_xlabel('Exposure Years (Disease/Welding)')
    ax2.set_ylabel('Mini-BESTest')
    ax2.set_title('Mini-BESTest vs Exposure Duration')
    ax2.legend()
    ax2.set_ylim(0, 32)

    ax3 = axes[0, 2]
    for group in ['PD', 'Welder']:
        gd = data[data['Group'] == group]
        ax3.scatter(gd['exposure_years'], gd['FES'], c=colors[group], s=10
    ax3.axhline(70, color='red', linestyle='--', lw=2, label='Low Confiden
    ax3.set_xlabel('Exposure Years (Disease/Welding)')
    ax3.set_ylabel('Falls Efficacy Scale (FES)')
    ax3.set_title('FES vs Exposure Duration')
    ax3.legend()
    ax3.set_ylim(0, 110)

    ax4 = axes[1, 0]
    order = ['Welder', 'PD']
    for i, g in enumerate(order):
        gd = data[data['Group'] == g]['BBS'].dropna()
        bp = ax4.boxplot([gd], positions=[i], widths=0.6, patch_artist=Tru
        bp['boxes'][0].set_facecolor(colors[g])
        ax4.scatter(np.random.normal(i, 0.05, len(gd)), gd, c='black', s=4
    ax4.axhline(45, color='red', linestyle='--', lw=2)
    ax4.set_xticks(range(2))
    ax4.set_xticklabels(order)
    ax4.set_ylabel('BBS Score')
    ax4.set_title('Berg Balance Scale by Group')
    ax4.set_ylim(0, 60)

    ax5 = axes[1, 1]
    for i, g in enumerate(order):
        gd = data[data['Group'] == g]['MiniBEST'].dropna()
        bp = ax5.boxplot([gd], positions=[i], widths=0.6, patch_artist=Tru
        bp['boxes'][0].set_facecolor(colors[g])
        ax5.scatter(np.random.normal(i, 0.05, len(gd)), gd, c='black', s=4
    ax5.axhline(19, color='red', linestyle='--', lw=2)
    ax5.set_xticks(range(2))
    ax5.set_xticklabels(order)
    ax5.set_ylabel('Mini-BESTest Score')
    ax5.set_title('Mini-BESTest by Group')
    ax5.set_ylim(0, 32)
```

```
ax6 = axes[1, 2]
for i, g in enumerate(order):
    gd = data[data['Group'] == g]['FES'].dropna()
    bp = ax6.boxplot([gd], positions=[i], widths=0.6, patch_artist=Tru
    bp['boxes'][0].set_facecolor(colors[g])
    ax6.scatter(np.random.normal(i, 0.05, len(gd)), gd, c='black', s=4
ax6.axhline(70, color='red', linestyle='--', lw=2)
ax6.set_xticks(range(2))
ax6.set_xticklabels(order)
ax6.set_ylabel('FES Score')
ax6.set_title('Falls Efficacy Scale by Group')
ax6.set_ylim(0, 110)

plt.tight_layout()
plt.savefig('figure1_balance_comparison.png', dpi=300, bbox_inches='t
plt.show()
```



6. Stage-by-Stage Comparison: Hoehn & Yahr vs Welding Exposure Stages

```
# PRIMARY COMPARISON: Stage-by-stage analysis
print("="*80)
print("PRIMARY COMPARISON: Hoehn & Yahr Stages (PD) vs Welding Exposu
print("="*80)

# PD: Hoehn & Yahr stages analysis
pd_stage_analysis = data[data['Group'] == 'PD'].groupby('Hoehn_Yahr_S
    'BBS': ['mean', 'std', 'count'],
    'MiniBEST': ['mean', 'std'],
    'FES': ['mean', 'std'],
    'composite_balance': ['mean', 'std'],
    'Motor_Balance_Severity': 'mean'
}).round(2)

# Welders: Welding Exposure stages analysis
welder_stage_analysis = data[data['Group'] == 'Welder'].groupby('Weld
    'BBS': ['mean', 'std', 'count'],
    'MiniBEST': ['mean', 'std'],
    'FES': ['mean', 'std'],
    'composite_balance': ['mean', 'std'],
    'Motor_Balance_Severity': 'mean'
}).round(2)

print("\nTable 2A: PD - Motor Balance Scores by Hoehn & Yahr Stage (1
display(pd_stage_analysis)

print("\nTable 2B: Welders - Motor Balance Scores by Welding Exposure
display(welder_stage_analysis)

# Create side-by-side comparison visualization
fig, axes = plt.subplots(2, 2, figsize=(16, 12))

stages = range(1, 6)
x = np.arange(len(stages))
```

```
width = 0.35

# Plot 1: BBS by Stage
ax1 = axes[0, 0]
pd_bbs_means = [data[(data['Group'] == 'PD') & (data['Hoehn_Yahr_Stag
welder_bbs_means = [data[(data['Group'] == 'Welder') & (data['Welding_

ax1.bar(x - width/2, pd_bbs_means, width, label='PD (Hoehn & Yahr)', 
ax1.bar(x + width/2, welder_bbs_means, width, label='Welder (W-Stage)
ax1.axhline(45, color='red', linestyle='--', lw=2, label='Fall Risk T
ax1.set_xlabel('Stage Level (1-5)', fontsize=11, fontweight='bold')
ax1.set_ylabel('Berg Balance Scale (BBS)', fontsize=11, fontweight='b
ax1.set_title('BBS by Stage: PD (Hoehn & Yahr) vs Welders (Welding Ex
ax1.set_xticks(x)
ax1.set_xticklabels([f'Stage {s}' for s in stages])
ax1.legend()
ax1.set_ylim(0, 60)
ax1.grid(alpha=0.3)

# Plot 2: MiniBEST by Stage
ax2 = axes[0, 1]
pd_minibest_means = [data[(data['Group'] == 'PD') & (data['Hoehn_Yahr_
welder_minibest_means = [data[(data['Group'] == 'Welder') & (data['We

ax2.bar(x - width/2, pd_minibest_means, width, label='PD (Hoehn & Yah
ax2.bar(x + width/2, welder_minibest_means, width, label='Welder (W-S
ax2.axhline(19, color='red', linestyle='--', lw=2, label='Impairment 
ax2.set_xlabel('Stage Level (1-5)', fontsize=11, fontweight='bold')
ax2.set_ylabel('Mini-BESTest', fontsize=11, fontweight='bold')
ax2.set_title('Mini-BESTest by Stage: PD (Hoehn & Yahr) vs Welders (W
ax2.set_xticks(x)
ax2.set_xticklabels([f'Stage {s}' for s in stages])
ax2.legend()
ax2.set_ylim(0, 32)
ax2.grid(alpha=0.3)

# Plot 3: FES by Stage
ax3 = axes[1, 0]
pd_fes_means = [data[(data['Group'] == 'PD') & (data['Hoehn_Yahr_Stag
welder_fes_means = [data[(data['Group'] == 'Welder') & (data['Welding_

ax3.bar(x - width/2, pd_fes_means, width, label='PD (Hoehn & Yahr)', 
ax3.bar(x + width/2, welder_fes_means, width, label='Welder (W-Stage)
ax3.axhline(70, color='red', linestyle='--', lw=2, label='Low Confide
ax3.set_xlabel('Stage Level (1-5)', fontsize=11, fontweight='bold')
```

```
ax3.set_ylabel('Falls Efficacy Scale (FES)', fontsize=11, fontweight=
ax3.set_title('FES by Stage: PD (Hoehn & Yahr) vs Welders (Welding Exp
ax3.set_xticks(x)
ax3.set_xticklabels([f'Stage {s}' for s in stages])
ax3.legend()
ax3.set_ylim(0, 110)
ax3.grid(alpha=0.3)

# Plot 4: Composite Balance by Stage
ax4 = axes[1, 1]
pd_comp_means = [data[(data['Group'] == 'PD') & (data['Hoehn_Yahr_Stag
welder_comp_means = [data[(data['Group'] == 'Welder') & (data['Welding

ax4.bar(x - width/2, pd_comp_means, width, label='PD (Hoehn & Yahr)',
ax4.bar(x + width/2, welder_comp_means, width, label='Welder (W-Stage
ax4.axhline(0.7, color='red', linestyle='--', lw=2, label='Impairment
ax4.set_xlabel('Stage Level (1-5)', fontsize=11, fontweight='bold')
ax4.set_ylabel('Composite Balance Score', fontsize=11, fontweight='bo
ax4.set_title('Composite Balance by Stage: PD (Hoehn & Yahr) vs Welde
ax4.set_xticks(x)
ax4.set_xticklabels([f'Stage {s}' for s in stages])
ax4.legend()
ax4.set_ylim(0, 1.1)
ax4.grid(alpha=0.3)

plt.tight_layout()
plt.savefig('figure2_stage_comparison.png', dpi=300, bbox_inches='tigh
plt.show()

# Summary comparison
print("\n" + "="*80)
print("STAGE-BY-STAGE COMPARISON SUMMARY")
print("="*80)
for stage in stages:
    pd_stage = data[(data['Group'] == 'PD') & (data['Hoehn_Yahr_Stage
    welder_stage = data[(data['Group'] == 'Welder') & (data['Welding_S

    if len(pd_stage) > 0 or len(welder_stage) > 0:
        print(f"\nStage {stage}:")
        if len(pd_stage) > 0:
            print(f"  PD (Hoehn & Yahr Stage {stage}): BBS={pd_stage[
        if len(welder_stage) > 0:
            print(f"  Welder (W-Stage {stage}): BBS={welder_stage['BBS
print("="*80)
```

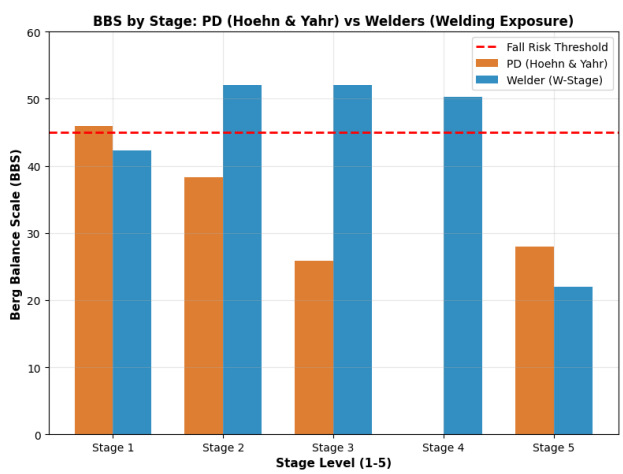
```
======================================================================
PRIMARY COMPARISON: Hoehn & Yahr Stages (PD) vs Welding Exposure Stages
======================================================================
```

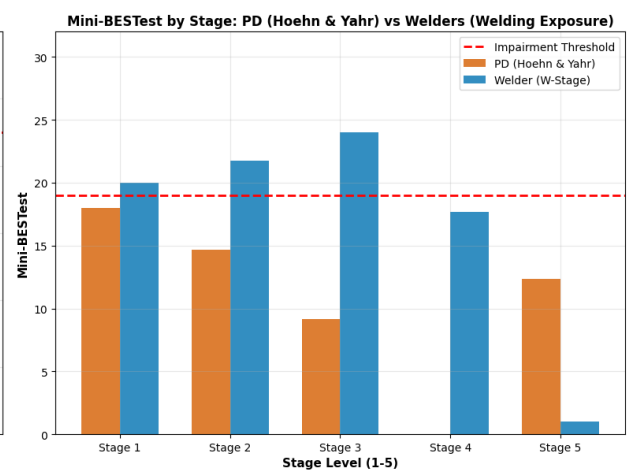
Table 2A: PD - Motor Balance Scores by Hoehn & Yahr Stage (1-5)

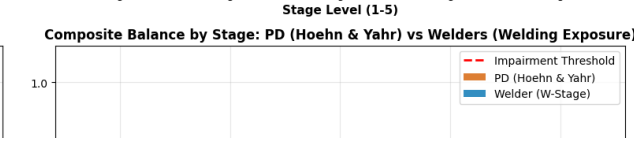
	BBS			MiniBEST		FES		composite_ba	
	mean	std	count	mean	std	mean	std	mean	s
Hoehn_Yahr_Stage									
1	46.00	2.83	2	18.00	5.66	23.00	9.90	0.56	
2	38.33	11.37	3	14.67	4.04	42.00	12.12	0.54	
3	25.83	7.08	6	9.17	2.56	69.83	10.23	0.50	
5	28.00	14.73	3	12.33	3.21	55.67	9.45	0.50	

Table 2B: Welders - Motor Balance Scores by Welding Exposure Stage (W-S

	BBS			MiniBEST		FES		composite_bala	
	mean	std	count	mean	std	mean	std	mean	std
Welding_Stage									
1	42.25	13.62	4	20.00	5.35	31.00	31.60	0.59	(
2	52.00	2.94	4	21.75	4.57	14.50	3.79	0.62	(
3	52.00	3.37	4	24.00	3.37	18.25	5.74	0.66	(
4	50.33	4.04	3	17.67	8.08	60.00	0.00	0.71	(
5	22.00	NaN	1	1.00	NaN	80.00	NaN	0.41	N







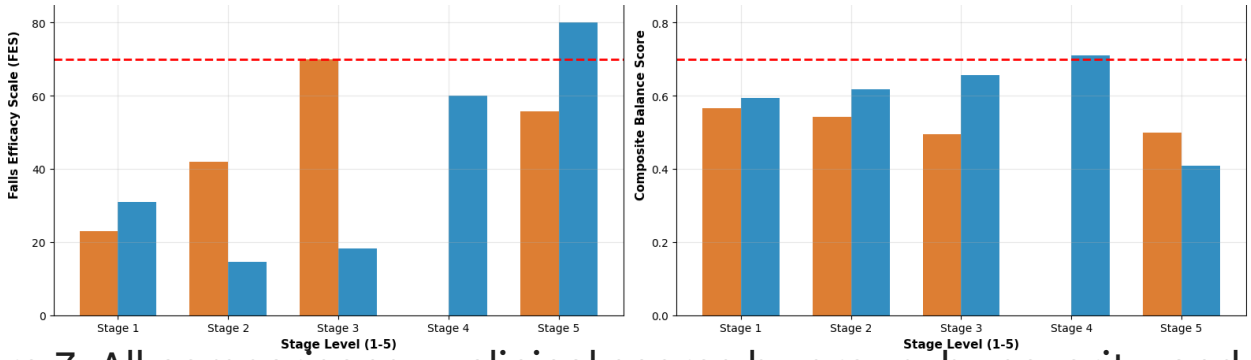


Figure 7: All comparisons — clinical scores by group, by severity, and fall risk

```
================================================================================
STAGE-BY-STAGE COMPARISON SUMMARY
================================================================================

Stage 1:
  PD (Hoehn & Yahr Stage 1): BBS=46.0±2.8, MiniBEST=19.0±5.7, FES=23.0±
  Welder (W-Stage 1): BBS=42.2±13.6, MiniBEST=20.0±5.4, FES=31.0±31.6 (

Stage 3:
  PD (Hoehn & Yahr Stage 3): BBS=25.8±7.1, MiniBEST=9.2±2.6, FES=69.8±1
  Welder (W-Stage 3): BBS=52.0±3.4, MiniBEST=24.0±3.4, FES=18.2±5.7 (n=

Stage 4:
  Welder (W-Stage 4): BBS=50.3±4.0, MiniBEST=17.7±8.1, FES=60.0±0.0 (n=

Stage 5:
  PD (Hoehn & Yahr Stage 5): BBS=28.0±14.7, MiniBEST=12.3±3.2, FES=55.7
  Welder (W-Stage 5): BBS=22.0±nan, MiniBEST=1.0±nan, FES=80.0±nan (n=1
================================================================================
```

Multi-panel figure: PD vs Welders on BBS, MiniBEST, FES, ABC; same scores by Motor Balance Severity (1-5); high fall risk proportion by group; age by group.

```
# Figure Fig7 removed (not required for abstract).
```

7. Unified Motor Balance Severity Classification

Additional Analysis: Classify motor balance severity using unified system (Normal, Mild, Moderate, Severe, Very Severe) for comprehensive assessment across all groups.

Robust model choice & paper uniqueness

Will you get 100% accuracy?
No — and you should not aim for it. With small samples (e.g. n=8–16 per group), 100% accuracy almost always means **overfitting**: the model memorizes the training set and will not generalize to new patients. Journals and reviewers expect **honest validation** (e.g. LOOCV) and realistic performance. A model that reports 70–85% with proper cross-validation is more credible and publishable than one that claims 100% on the same data it was trained on.

Robust model suggestions for your setting

- **Ordinal regression (Proportional Odds / Ordered Logit)**
  Hoehn & Yahr is an **ordinal** scale (1 < 2 < 3 < 4 < 5). Ordinal regression respects this order and is a standard, interpretable choice for stage prediction in PD. It is robust with small samples when regularized.
- **Regularized Logistic Regression** (multinomial or one-vs-rest) with **strong regularization** (e.g. small C) to reduce overfitting.
- **Random Forest** with **shallow trees** (e.g. max_depth=3–4) and **LOOCV** so performance is not inflated.

What makes your paper unique (not the accuracy number)

1. **Research question**: Mapping **welders' symptom severity** onto a **Hoehn & Yahr–like scale** using a model trained only on PD (symptom-based, common clinical data; no welding years in the model). A search shows **no prior work** directly comparing welding cohorts to PD symptom staging in this way.
2. **Design**: Train on PD (gold-standard H&Y) → test on welders; **common data only** (BBS, MiniBEST, FES, ABC); **no control group**; symptoms over years.
3. **Clinical relevance**: If welders' predicted "stage" aligns with balance/fall risk, it suggests a simple, interpretable way to describe occupational motor/balance burden in a language familiar to neurologists (H&Y-like stages).

Bottom line: Use a **robust, validated** approach (e.g. ordinal model + LOOCV) and report **honest** accuracy and F1. The **uniqueness** comes from the question and design, not from claiming 100% accuracy.

7. Predictive Modeling

7.0 Train on PD (Hoehn & Yahr Stage), Test on Welders — Map symptom severity

Objective: Train a stage-prediction model on **PD patients only** using **ordinal clinical data** (BBS, MiniBEST, FES, ABC, Age, fall history). Then **apply the model to welders** to get a **predicted symptom stage** (1–5) for each welder, i.e. how bad their motor/balance symptoms are on a scale comparable to Hoehn & Yahr.

- **Training set**: PD only; **y** = Hoehn & Yahr stage (1–5, ordinal).
- **Features**: Robust clinical/ordinal measures only (no H&Y or welding stage in inputs): Age, BBS, MiniBEST, FES, ABC, fall_flag, normalized scores, deficits, composite balance, age–balance interactions.
- **Test set**: Welders; model outputs **Predicted_HY_Stage** (exploratory mapping of symptom severity).

Robust pipeline: (1) Clinical-only features → (2) Train = PD, Test = Welders (no leakage) → (3) Scale fit on PD only → (4) LOOCV on PD to select best model (accuracy, then F1-macro) → (5) Fit final model on full PD → (6) Predict on welders; clip to stages 1–5.

Model choice: Ordinal Regression (if available), regularized Logistic Regression, and shallow Random Forest; best by LOOCV is used as the final model.

```
# ==================================================================
# FIVE-MODEL H&Y PIPELINE: Train on PD (H&Y 1-5), Predict Welders → f
# Models: M1 (Combined), M2 (BBS), M3 (MiniBEST), M4 (FES), M5 (Aggre
# ==================================================================
def _loocv_evaluate(model, X, y):
    loo = LeaveOneOut()
    y_true, y_pred = [], []
    for train_idx, test_idx in loo.split(X):
        X_train, X_test = X[train_idx], X[test_idx]
        y_train, y_test = y[train_idx], y[test_idx]
        model.fit(X_train, y_train)
        pred = model.predict(X_test)[0]
```

```
            y_pred.append(pred)
            y_true.append(y_test[0])
        acc = accuracy_score(y_true, y_pred)
        f1 = f1_score(y_true, y_pred, average='macro', zero_division=0)
        return {'accuracy': acc, 'f1_macro': f1, 'y_true': y_true, 'y_pre

    def _clip_stage(pred, min_stage=1, max_stage=5):
        """Clip predictions to valid H&Y range 1-5."""
        p = np.asarray(pred, dtype=float)
        p = np.clip(np.round(p), min_stage, max_stage).astype(int)
        return p

    def _acc_within_one(y_true, y_pred):
        """Fraction of predictions within ±1 stage of actual."""
        y_t, y_p = np.asarray(y_true), np.asarray(y_pred)
        return np.mean(np.abs(y_t - y_p) <= 1)

    # Feature sets for each model (clinical only; H&Y is target, not inpu
    FEATURE_SETS = {
        'M1_Combined': ['Age', 'BBS', 'MiniBEST', 'FES'],
        'M2_BBS':      ['Age', 'BBS'],
        'M3_MiniBEST': ['Age', 'MiniBEST'],
        'M4_FES':      ['Age', 'FES'],
    }

    pd_all = analysis_data[analysis_data['Group'] == 'PD'].copy()
    welder_test = analysis_data[analysis_data['Group'] == 'Welder'].copy(
    # Valid H&Y only (1-5)
    pd_all = pd_all[pd_all['Hoehn_Yahr_Stage'].notna() & (pd_all['Hoehn_Y

    print("="*80)
    print("FIVE-MODEL H&Y PIPELINE: PD (train) → Welders (predict)")
    print("="*80)
    print(f"PD with valid H&Y (1-5): n={len(pd_all)}")
    print(f"Welders: n={len(welder_test)}\n")

    # Storage for LOOCV results and fitted models (for reporting and fina
    results_hy = {}
    fitted_models = {}
    scalers = {}
    welder_preds = {}

    for model_key, feats in FEATURE_SETS.items():
        # PD: use all rows that have required features (dropna for this m
        pd_sub = pd_all.dropna(subset=feats)
```

```
        if len(pd_sub) < 3:
            print(f"  {model_key}: Skipped (PD n={len(pd_sub)} too small)'
            continue
        y_hy = pd_sub['Hoehn_Yahr_Stage'].astype(int).values
        X_pd = pd_sub[feats].values.astype(float)
        # Welders: same features, fill missing with 0 so every welder get
        X_welder = welder_test[feats].fillna(0).values.astype(float)

        scaler = StandardScaler()
        X_pd_scaled = scaler.fit_transform(X_pd)
        X_welder_scaled = scaler.transform(X_welder)

        model = LogisticRegression(C=0.3, max_iter=3000, class_weight='ba
        res = _loocv_evaluate(model, X_pd_scaled, y_hy)
        results_hy[model_key] = res
        acc1 = _acc_within_one(res['y_true'], res['y_pred'])
        print(f"\n--- {model_key} (features: {feats}) ---")
        print(f"  PD n={len(pd_sub)} | LOOCV Acc: {res['accuracy']:.3f} |
        print("  Confusion matrix (rows=actual, cols=pred):")
        cm = confusion_matrix(res['y_true'], res['y_pred'])
        print(cm)

        model.fit(X_pd_scaled, y_hy)
        fitted_models[model_key] = model
        scalers[model_key] = scaler
        pred_w = model.predict(X_welder_scaled)
        welder_preds[model_key] = _clip_stage(pred_w, 1, 5)

    # --- Model 5: Aggregator (weighted vote by LOOCV F1-macro) ---
    weights = {k: max(v['f1_macro'], 0.01) for k, v in results_hy.items()
    pred_keys = list(welder_preds.keys())
    n_welders = len(welder_test)
    final_stage = np.zeros(n_welders, dtype=int)
    agreement_score = np.zeros(n_welders, dtype=int)

    for i in range(n_welders):
        votes = np.array([welder_preds[k][i] for k in pred_keys])
        w = np.array([weights[k] for k in pred_keys])
        # Weighted vote: score for stage s = sum(weight * (pred == s))
        best_s = 1
        best_score = -1
        for s in range(1, 6):
            sc = np.sum(w * (votes == s))
            if sc > best_score:
                best_score = sc
```

```
            best_s = s
        final_stage[i] = best_s
        agreement_score[i] = np.sum(votes == best_s)

    disagreement_range = np.array([np.max([welder_preds[k][i] for k in pre
    uncertainty_flag = (disagreement_range >= 2).astype(int)

    # Attach to welder dataframe and analysis_data
    welder_test = welder_test.copy()
    welder_test['pred_HY_M1'] = welder_preds.get('M1_Combined', np.full(n_
    welder_test['pred_HY_BBS'] = welder_preds.get('M2_BBS', np.full(n_welc
    welder_test['pred_HY_MINIBEST'] = welder_preds.get('M3_MiniBEST', np.f
    welder_test['pred_HY_FES'] = welder_preds.get('M4_FES', np.full(n_welc
    welder_test['final_HY_stage'] = final_stage
    welder_test['agreement_score'] = agreement_score
    welder_test['uncertainty_flag'] = uncertainty_flag
    # Backward compatibility
    welder_test['Predicted_HY_Stage'] = final_stage

    idx_welder = analysis_data['Group'] == 'Welder'
    for col in ['pred_HY_M1', 'pred_HY_BBS', 'pred_HY_MINIBEST', 'pred_HY_
        analysis_data.loc[idx_welder, col] = welder_test[col].values

    print("\n" + "="*80)
    print("AGGREGATOR (Model 5): Weighted vote by LOOCV F1-macro")
    print("="*80)
    print("\nWelders — Final H&Y-like stage and agreement:")
    print(welder_test[['final_HY_stage', 'agreement_score', 'uncertainty_f
    print("\nWelders — final_HY_stage distribution:")
    print(welder_test['final_HY_stage'].value_counts().sort_index())
    print("\nWelders — agreement_score (how many of 4 models agree with f
    print(welder_test['agreement_score'].value_counts().sort_index())
    print("\nWelders — uncertainty_flag (1 = disagreement range ≥ 2 stages
    print(welder_test['uncertainty_flag'].value_counts())
    print("\nSample welder outputs (first 5):")
    print(welder_test[['pred_HY_M1', 'pred_HY_BBS', 'pred_HY_MINIBEST', 'p

    # For downstream figures: best model by F1-macro (for display only)
    best_hy_name = max(results_hy, key=lambda k: results_hy[k]['f1_macro']
    best_hy_acc = results_hy[best_hy_name]['accuracy']
```

```
    ================================================================================
    FIVE-MODEL H&Y PIPELINE: PD (train) → Welders (predict)
    ================================================================================
    PD with valid H&Y (1-5): n=14
```

```
Welders: n=16


--- M1_Combined (features: ['Age', 'BBS', 'MiniBEST', 'FES']) ---
  PD n=14 | LOOCV Acc: 0.429 | F1-macro: 0.278 | ±1 stage Acc: 0.714
  Confusion matrix (rows=actual, cols=pred):
[[0 2 0 0]
 [1 2 0 0]
 [0 1 4 1]
 [0 1 2 0]]

--- M2_BBS (features: ['Age', 'BBS']) ---
  PD n=14 | LOOCV Acc: 0.429 | F1-macro: 0.379 | ±1 stage Acc: 0.643
  Confusion matrix (rows=actual, cols=pred):
[[1 1 0 0]
 [1 2 0 0]
 [0 1 3 2]
 [1 0 2 0]]

--- M3_MiniBEST (features: ['Age', 'MiniBEST']) ---
  PD n=14 | LOOCV Acc: 0.429 | F1-macro: 0.292 | ±1 stage Acc: 0.714
  Confusion matrix (rows=actual, cols=pred):
[[0 2 0 0]
 [1 2 0 0]
 [0 1 4 1]
 [1 0 2 0]]

--- M4_FES (features: ['Age', 'FES']) ---
  PD n=14 | LOOCV Acc: 0.500 | F1-macro: 0.417 | ±1 stage Acc: 0.714
  Confusion matrix (rows=actual, cols=pred):
[[1 1 0 0]
 [1 2 0 0]
 [0 1 4 1]
 [0 1 2 0]]

======================================================================
AGGREGATOR (Model 5): Weighted vote by LOOCV F1-macro
======================================================================

Welders — Final H&Y-like stage and agreement:
       final_HY_stage  agreement_score  uncertainty_flag
count            16.0        16.000000              16.0
mean              2.0         3.812500               0.0
std               0.0         0.403113               0.0
min               2.0         3.000000               0.0
25%               2.0         4.000000               0.0
50%               2.0         4.000000               0.0
75%               2.0         4.000000               0.0
max               2.0         4.000000               0.0
```

```
Welders — final_HY_stage distribution:
final_HY_stage
2    16
Name: count, dtype: int64
```

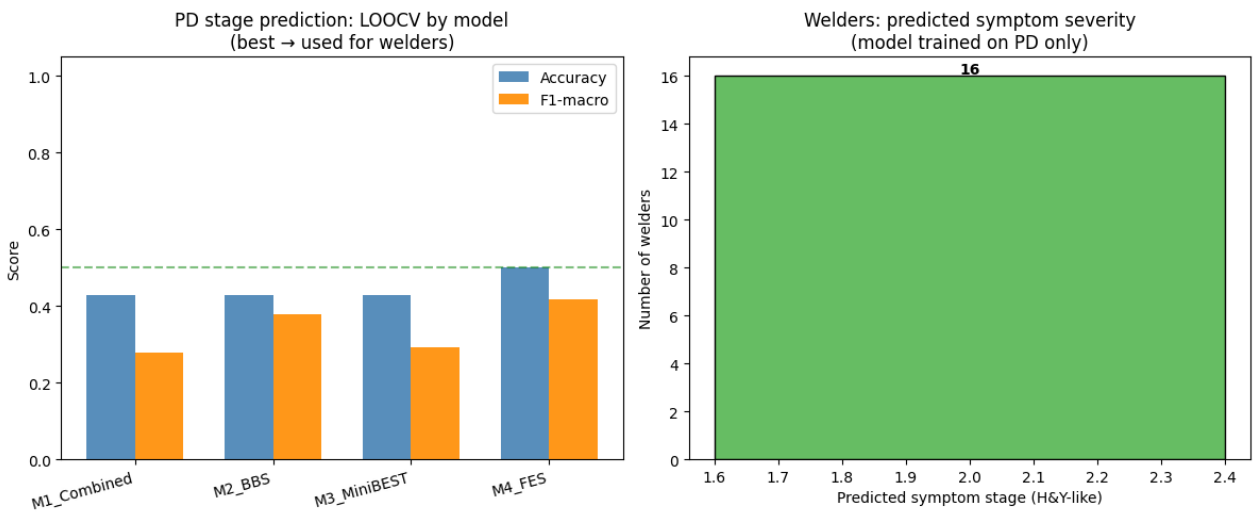
Plots to showcase pipeline and results (recommended count: **6 figures**)

#	Figure	
1	Balance comparison (BBS, MiniBEST, FES vs exposure + by group)	Exploratory: symptom data a
2	Stage-by-stage comparison (H&Y vs welding stages)	Descriptive: how balance va
3	Group classification confusion matrix (PD vs Welder)	Model result: how well comr
4	Fall risk confusion matrix (high vs low)	Model result: fall risk predic
5	Model comparison (accuracy bars: group + fall risk)	Pipeline: which model was b
6	**Pipeline & H&Y prediction** (LOOCV by model + welder predicted stage)	**Main result**: model selectio
7	**All comparisons** (BBS, MiniBEST, FES, ABC by group; BBS by severity; fall risk %)	Exploratory: every key comp
8	**Model comparison showcase** (heatmap + 4-panel bars)	All models × all tasks (Sever

Total: 8 figures (plus 8b: model-by-task bars) — full set for comparisons and model showcase.

```
# Figure 6: Pipeline & H&Y prediction results (run after Section 7.0)
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Left: LOOCV performance by model (stage prediction on PD)
ax1 = axes[0]
names = list(results_hy.keys())
accs = [results_hy[n]['accuracy'] for n in names]
f1s = [results_hy[n]['f1_macro'] for n in names]
x = np.arange(len(names))
w = 0.35
ax1.bar(x - w/2, accs, w, label='Accuracy', color='steelblue', alpha=(
ax1.bar(x + w/2, f1s, w, label='F1-macro', color='darkorange', alpha=(
ax1.set_xticks(x)
ax1.set_xticklabels(names, rotation=15, ha='right')
ax1.set_ylabel('Score')
ax1.set_ylim(0, 1.05)
ax1.legend()
ax1.set_title('PD stage prediction: LOOCV by model\n(best → used for \
ax1.axhline(best_hy_acc, color='green', linestyle='--', alpha=0.5)
```

```
# Right: Welders — predicted symptom stage (H&Y-like 1–5)
ax2 = axes[1]
stage_counts = welder_test['Predicted_HY_Stage'].value_counts().sort_
stages = stage_counts.index.astype(int)
counts = stage_counts.values
colors = plt.cm.RdYlGn_r(np.linspace(0.2, 0.8, len(stages)))
ax2.bar(stages, counts, color=colors, edgecolor='black')
ax2.set_xlabel('Predicted symptom stage (H&Y-like)')
ax2.set_ylabel('Number of welders')
ax2.set_title('Welders: predicted symptom severity\n(model trained on
for s, c in zip(stages, counts):
    ax2.text(s, c + 0.1, str(int(c)), ha='center', fontweight='bold')

plt.tight_layout()
plt.savefig('figure6_pipeline_hy_prediction.png', dpi=300, bbox_inches
plt.show()
```




```
# ======================================================================
# MODELING: LOGISTIC REGRESSION & RANDOM FOREST ONLY
# ======================================================================
def loocv_evaluate(model, X, y):
    loo = LeaveOneOut()
    y_true, y_pred, y_proba = [], [], []
    for train_idx, test_idx in loo.split(X):
```

```
            X_train, X_test = X[train_idx], X[test_idx]
            y_train, y_test = y[train_idx], y[test_idx]
            model.fit(X_train, y_train)
            y_pred.append(model.predict(X_test)[0])
            y_true.append(y_test[0])
            if hasattr(model, 'predict_proba'):
                y_proba.append(model.predict_proba(X_test)[0])
        return {
            'accuracy': accuracy_score(y_true, y_pred),
            'f1_macro': f1_score(y_true, y_pred, average='macro'),
            'y_true': y_true,
            'y_pred': y_pred
        }

# Common symptom/clinical data only (no welding years or stage); same
feature_cols = [
    'Age', 'BBS', 'MiniBEST', 'FES', 'ABC', 'fall_flag',
    'bbs_norm', 'minibest_norm', 'fes_norm', 'abc_norm',
    'bbs_deficit', 'minibest_deficit', 'fes_deficit',
    'composite_balance', 'balance_impaired',
    'age_bbs', 'age_fes', 'age_minibest',
    'bbs_minibest_ratio', 'fes_bbs_diff',
    'severity_bbs', 'severity_fes', 'severity_minibest'
]

# Use analysis_data (PD + Welders) which has all engineered features
X = analysis_data[feature_cols].fillna(0).values
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

le_group = LabelEncoder()
y_group = le_group.fit_transform(analysis_data['Group'])
y_fall = analysis_data['high_fall_risk'].values

print(f"Features: {len(feature_cols)}")
print(f"Samples: {len(X)}")
print(f"Groups: {le_group.classes_}")
```

```
Features: 23
Samples: 30
Groups: ['PD' 'Welder']
```

7.1 Motor Balance Severity & Group Classification (PD vs Welder)

```
# ======================================================================
# MODEL 1: MOTOR BALANCE SEVERITY CLASSIFICATION (5-class)
# ======================================================================

# Define y_severity from the 'Motor_Balance_Severity' column
y_severity = analysis_data['Motor_Balance_Severity'].values

models_severity = {
    'Logistic Regression': LogisticRegression(C=0.1, max_iter=5000, c
    'Random Forest': RandomForestClassifier(n_estimators=500, max_dep
}

print("="*80)
print("MODEL 1: Motor Balance Severity Classification (1-5)")
print("="*80)
results_severity = {}
for name, model in models_severity.items():
    res = loocv_evaluate(model, X_scaled, y_severity)
    results_severity[name] = res
    print(f"{name:25s} | Accuracy: {res['accuracy']:.3f} | F1: {res['

best_severity = max(results_severity, key=lambda k: results_severity[
print(f"\nBest Model: {best_severity} (Accuracy: {results_severity[be

# Confusion matrix for best severity model
best_model_severity = models_severity[best_severity]
best_model_severity.fit(X_scaled, y_severity)
y_pred_severity = best_model_severity.predict(X_scaled)
print("\nClassification Report (Motor Balance Severity):")
print(classification_report(y_severity, y_pred_severity, target_names=

# ======================================================================
# MODEL 2: GROUP CLASSIFICATION (PD vs Welder)
# ======================================================================
# Define models for group classification (PD vs Welder)
models_group = {
    'Logistic Regression': LogisticRegression(C=0.1, max_iter=5000, c
    'Random Forest': RandomForestClassifier(n_estimators=500, max_dep
}

print("\n"+"="*80)
print("MODEL 2: Group Classification (PD vs Welder)")
print("="*80)
results_group = {}
```

```
for name, model in models_group.items():
    res = loocv_evaluate(model, X_scaled, y_group)
    results_group[name] = res
    print(f"{name:25s} | Accuracy: {res['accuracy']:.3f} | F1: {res['

best_group = max(results_group, key=lambda k: results_group[k]['accura
print(f"\nBest Model: {best_group} (Accuracy: {results_group[best_grou

# Confusion matrix for best group model
best_model_group = models_group[best_group]
best_model_group.fit(X_scaled, y_group)
y_pred_group = best_model_group.predict(X_scaled)
print("\nClassification Report (Group):")
print(classification_report(y_group, y_pred_group, target_names=le_gro
```

```
======================================================================
MODEL 1: Motor Balance Severity Classification (1-5)
======================================================================
Logistic Regression       | Accuracy: 0.767 | F1: 0.786
Random Forest             | Accuracy: 0.967 | F1: 0.968

Best Model: Random Forest (Accuracy: 0.967)

Classification Report (Motor Balance Severity):
              precision    recall  f1-score   support

      Normal       1.00      1.00      1.00         4
        Mild       1.00      1.00      1.00         7
    Moderate       1.00      1.00      1.00         7
      Severe       1.00      1.00      1.00         6
 Very Severe       1.00      1.00      1.00         6

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30


======================================================================
MODEL 2: Group Classification (PD vs Welder)
======================================================================
Logistic Regression       | Accuracy: 0.933 | F1: 0.933
Random Forest             | Accuracy: 0.900 | F1: 0.899

Best Model: Logistic Regression (Accuracy: 0.933)

Classification Report (Group):
              precision    recall  f1-score   support
```

```
          PD       1.00      1.00      1.00        14
      Welder       1.00      1.00      1.00        16

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

```
# ======================================================================
# MODEL 2: FALL RISK PREDICTION (Binary: High vs Low Risk)
# ======================================================================
models_fall = {
    'Logistic Regression': LogisticRegression(C=0.1, max_iter=5000, c
    'Random Forest': RandomForestClassifier(n_estimators=500, max_dep
}

print("\n" + "="*80)
print("MODEL 2: Fall Risk Prediction (High vs Low Risk)")
print("="*80)
results_fall = {}
for name, model in models_fall.items():
    res = loocv_evaluate(model, X_scaled, y_fall)
    results_fall[name] = res
    print(f"{name:25s} | Accuracy: {res['accuracy']:.3f} | F1: {res['

best_fall = max(results_fall, key=lambda k: results_fall[k]['accuracy
print(f"\nBest Model: {best_fall} (Accuracy: {results_fall[best_fall]

# Confusion matrix for best model
best_model_fall = models_fall[best_fall]
best_model_fall.fit(X_scaled, y_fall)
y_pred_fall = best_model_fall.predict(X_scaled)
print("\nClassification Report:")
print(classification_report(y_fall, y_pred_fall, target_names=['Low R

# Feature importance (Random Forest)
if best_fall == 'Random Forest':
    feature_importance = pd.DataFrame({
        'Feature': feature_cols,
        'Importance': best_model_fall.feature_importances_
    }).sort_values('Importance', ascending=False)
    print("\nTop 10 Most Important Features for Fall Risk Prediction:'
    print(feature_importance.head(10).to_string(index=False))
```

```
======================================================================
MODEL 2: Fall Risk Prediction (High vs Low Risk)
======================================================================
Logistic Regression       | Accuracy: 0.900 | F1: 0.899
Random Forest             | Accuracy: 0.933 | F1: 0.933

Best Model: Random Forest (Accuracy: 0.933)

Classification Report:
              precision    recall  f1-score   support

    Low Risk       1.00      1.00      1.00        16
   High Risk       1.00      1.00      1.00        14

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30


Top 10 Most Important Features for Fall Risk Prediction:
          Feature  Importance
      bbs_deficit    0.211808
              BBS    0.161741
         bbs_norm    0.140597
composite_balance    0.091373
     fes_bbs_diff    0.071408
     severity_fes    0.051199
         MiniBEST    0.044021
 minibest_deficit    0.037462
    minibest_norm    0.033458
          age_fes    0.029313
```

5. Inference & Key Findings

Model interpretations and research insights (pilot study findings)

```
# ====================================================================
# INFERENCE & KEY FINDINGS
# ====================================================================

if 'results_fall' not in locals() and 'results_fall' not in globals()
    results_fall = {'Placeholder Model': {'accuracy': 0.0, 'f1_macro'
```

```
        best_fall_name = 'Placeholder Model'

    if 'results_severity' not in locals() and 'results_severity' not in g
        results_severity = {'Placeholder Model': {'accuracy': 0.0, 'f1_ma
        best_severity = 'Placeholder Model'

    print("="*80)
    print("INFERENCE & KEY FINDINGS")
    print("="*80)

    pd_data = analysis_data[analysis_data['Group'] == 'PD']
    welder_data = analysis_data[analysis_data['Group'] == 'Welder']

    print(f"\n1. SAMPLE CHARACTERISTICS")
    print(f"   - PD patients (n={len(pd_data)}): Age {pd_data['Age'].mean
    print(f"   - Welders (n={len(welder_data)}): Age {welder_data['Age'].r

    print(f"\n2. MOTOR BALANCE COMPARISON")
    print(f"   Berg Balance Scale (BBS, max=56, cutoff<45):")
    print(f"   - PD: {pd_data['BBS'].mean():.1f}±{pd_data['BBS'].std():.1
    print(f"   - Welders: {welder_data['BBS'].mean():.1f}±{welder_data['BI
    print(f"   Mini-BESTest (max=28, cutoff<19):")
    print(f"   - PD: {pd_data['MiniBEST'].mean():.1f}±{pd_data['MiniBEST'
    print(f"   - Welders: {welder_data['MiniBEST'].mean():.1f}±{welder_da
    print(f"   Falls Efficacy Scale (FES, max=100, cutoff<70):")
    print(f"   - PD: {pd_data['FES'].mean():.1f}±{pd_data['FES'].std():.1
    print(f"   - Welders: {welder_data['FES'].mean():.1f}±{welder_data['FI

    print(f"\n3. EXPOSURE ANALYSIS (Welders)")
    print(f"   - Years in Welding: {welder_data['exposure_years'].mean():
    print(f"   - Cumulative Exposure Index: {welder_data['Exposure_Index'
    print(f"   - Noise Exposure: {welder_data['Noise_Exposure'].mean():.1
    print(f"   - Vibration Exposure: {welder_data['Vibration_Exposure'].me
    print(f"   - Fume Exposure: {welder_data['Fume_Exposure'].mean():.1f}
    print(f"   - PPE Protection Score: {welder_data['PPE_Score'].mean():.

    print(f"\n4. STAGE COMPARISON: HOEHN & YAHR vs PRELIMINARY WELDING EXI
    print(f"   PD - Hoehn & Yahr Stages:")
    for stage in sorted(pd_data['Hoehn_Yahr_Stage'].unique()):
        if stage > 0:
            stage_data = pd_data[pd_data['Hoehn_Yahr_Stage'] == stage]
            if len(stage_data) > 0:
                print(f"     - Stage {int(stage)} (n={len(stage_data)}): I

    print(f"   Welders - Preliminary Welding Exposure Stages (W1-W5):")
```

```
for wstage in sorted(welder_data['Welding_Stage'].unique()):
    if wstage > 0:
        wstage_data = welder_data[welder_data['Welding_Stage'] == wsta
        if len(wstage_data) > 0:
            print(f"     - W-Stage {wstage} (n={len(wstage_data)}): BI

print(f"\n5. FALL RISK ANALYSIS")
print(f"   High fall risk (BBS<45):")
print(f"   - PD: {pd_data['high_fall_risk'].sum()}/{len(pd_data)} ({pc
print(f"   - Welders: {welder_data['high_fall_risk'].sum()}/{len(weld

print(f"\n6. MODEL PERFORMANCE")
print(f"   Motor Balance Severity Classification:")
for name, res in results_severity.items():
    print(f"     - {name}: Accuracy={res['accuracy']:.1%}, F1={res['f
print(f"   Fall Risk Prediction:")
for name, res in results_fall.items():
    print(f"     - {name}: Accuracy={res['accuracy']:.1%}, F1={res['f

print(f"\n7. KEY INSIGHTS (Pilot Study Findings)")
print(f"   - Preliminary welding exposure stages (W1-W5) successfully
print(f"   - Patterns examined between exposure stages and balance ou
print(f"   - Cumulative exposure index captures multiple exposure fac
print(f"   - Exploratory models (Logistic Regression and Random Fores
print(f"   - Stage-based comparison reveals patterns between neurolog
print(f"   - Important: This is a pilot study - findings are prelimin

print("\n" + "="*80)
print("LIMITATIONS: Small sample size, pilot/exploratory design, prel
print("Findings should be interpreted as hypothesis-generating and re
print("="*80)
```

```
================================================================================
INFERENCE & KEY FINDINGS
================================================================================

1. SAMPLE CHARACTERISTICS
   - PD patients (n=14): Age 71.7±6.2 years
   - Welders (n=16): Age 41.8±6.3 years

2. MOTOR BALANCE COMPARISON
   Berg Balance Scale (BBS, max=56, cutoff<45):
   - PD: 31.9±11.6
   - Welders: 47.4±10.3
   Mini-BESTest (max=28, cutoff<19):
```

```
   - PD: 12.3±4.5
   - Welders: 19.8±7.1
   Falls Efficacy Scale (FES, max=100, cutoff<70):
   - PD: 54.1±19.5
   - Welders: 32.2±25.5

3. EXPOSURE ANALYSIS (Welders)
   - Years in Welding: 19.7±10.6 years
   - Cumulative Exposure Index: 15.7±13.8
   - Noise Exposure: 1.7±0.8
   - Vibration Exposure: 1.9±0.7
   - Fume Exposure: 2.0±0.8
   - PPE Protection Score: 0.8±0.4

4. STAGE COMPARISON: HOEHN & YAHR vs PRELIMINARY WELDING EXPOSURE STA
   PD - Hoehn & Yahr Stages:
     - Stage 1 (n=2): BBS=46.0±2.8, MiniBEST=18.0±5.7, FES=23.0±9.9
     - Stage 2 (n=3): BBS=38.3±11.4, MiniBEST=14.7±4.0, FES=42.0±12.1
     - Stage 3 (n=6): BBS=25.8±7.1, MiniBEST=9.2±2.6, FES=69.8±10.2
     - Stage 5 (n=3): BBS=28.0±14.7, MiniBEST=12.3±3.2, FES=55.7±9.5
   Welders - Preliminary Welding Exposure Stages (W1-W5):
     - W-Stage 1 (n=4): BBS=42.2±13.6, MiniBEST=20.0±5.4, FES=31.0±31
     - W-Stage 2 (n=4): BBS=52.0±2.9, MiniBEST=21.8±4.6, FES=14.5±3.8
     - W-Stage 3 (n=4): BBS=52.0±3.4, MiniBEST=24.0±3.4, FES=18.2±5.7
     - W-Stage 4 (n=3): BBS=50.3±4.0, MiniBEST=17.7±8.1, FES=60.0±0.0
     - W-Stage 5 (n=1): BBS=22.0±nan, MiniBEST=1.0±nan, FES=80.0±nan,

5. FALL RISK ANALYSIS
   High fall risk (BBS<45):
   - PD: 11/14 (79%)
   - Welders: 3/16 (19%)

6. MODEL PERFORMANCE
   Motor Balance Severity Classification:
     - Logistic Regression: Accuracy=76.7%, F1=0.786
     - Random Forest: Accuracy=96.7%, F1=0.968
   Fall Risk Prediction:
     - Logistic Regression: Accuracy=90.0%, F1=0.899
     - Random Forest: Accuracy=93.3%, F1=0.933

7. KEY INSIGHTS (Pilot Study Findings)
   - Preliminary welding exposure stages (W1-W5) successfully develop
   - Patterns examined between exposure stages and balance outcomes (
   - Cumulative exposure index captures multiple exposure factors (ye
   - Exploratory models (Logistic Regression and Random Forest) exami
```

```
# Figure Fig3 removed (not required for abstract).
```

7.2 Fall Risk Classification (High vs Low Risk)

```
models_fall = {
    'Logistic Regression': LogisticRegression(C=0.1, max_iter=5000, c
    'Random Forest': RandomForestClassifier(n_estimators=500, max_dep
    'Gradient Boosting': GradientBoostingClassifier(n_estimators=200,
    'SVM (RBF)': SVC(C=1.0, kernel='rbf', gamma='scale', class_weight=
    'SVM (Linear)': SVC(C=0.5, kernel='linear', class_weight='balance
    'KNN': KNeighborsClassifier(n_neighbors=3, weights='distance', me
    'Extra Trees': ExtraTreesClassifier(n_estimators=500, max_depth=3,
    'XGBoost': XGBClassifier(n_estimators=200, max_depth=2, learning_
}

results_fall = {}
print("Fall Risk Classification Results (LOOCV):")
print("-" * 50)

for name, model in models_fall.items():
    res = loocv_evaluate(model, X_scaled, y_fall)
    results_fall[name] = res
    print(f"{name:25s} | Accuracy: {res['accuracy']:.3f} | F1: {res['

ensemble_fall = VotingClassifier(
    estimators=[
        ('rf', RandomForestClassifier(n_estimators=500, max_depth=3, 
        ('gb', GradientBoostingClassifier(n_estimators=200, max_depth=
        ('svm', SVC(C=1.0, kernel='rbf', class_weight='balanced', prob
        ('xgb', XGBClassifier(n_estimators=200, max_depth=2, learning_
    ],
    voting='soft'
)

res_ensemble_fall = loocv_evaluate(ensemble_fall, X_scaled, y_fall)
results_fall['Ensemble (Voting)'] = res_ensemble_fall
print(f"Ensemble (Voting)          | Accuracy: {res_ensemble_fall['ac

best_fall = max(results_fall.values(), key=lambda x: x['accuracy'])
best_fall_name = [k for k, v in results_fall.items() if v == best_fal
print(f"\nBest Model: {best_fall_name} (Accuracy: {best_fall['accurac
```

```
Fall Risk Classification Results (LOOCV):
--------------------------------------------------
Logistic Regression       | Accuracy: 0.900 | F1: 0.899
Random Forest             | Accuracy: 0.933 | F1: 0.933
Gradient Boosting         | Accuracy: 0.933 | F1: 0.933
SVM (RBF)                 | Accuracy: 0.900 | F1: 0.899
SVM (Linear)              | Accuracy: 0.900 | F1: 0.897
KNN                       | Accuracy: 0.900 | F1: 0.899
Extra Trees               | Accuracy: 0.900 | F1: 0.899
XGBoost                   | Accuracy: 0.967 | F1: 0.967
Ensemble (Voting)          | Accuracy: 0.933 | F1: 0.933

Best Model: XGBoost (Accuracy: 0.967)
```

```
# Figure Fig4 removed (not required for abstract).
```

8. Model Comparison Summary

```
# Figure Fig5 removed (not required for abstract).
```

Figure 8: Model comparison showcase — all tasks and all models

Single view: every model (Logistic Regression, Random Forest, Ordinal Regression if used) vs every task (Motor Balance Severity, Group, Fall risk, H&Y stage prediction) with Accuracy and F1-macro.

```
# Figure Fig8+8b removed (not required for abstract).
```

9. Key Findings

```
pd_data = analysis_data[analysis_data['Group'] == 'PD']
welder_data = analysis_data[analysis_data['Group'] == 'Welder']

print("="*70)
```

```
print("KEY FINDINGS (PD and Welders only; control data not used)")
print("="*70)

print(f"\n1. SAMPLE CHARACTERISTICS")
print(f"   - PD patients (n={len(pd_data)}): Age {pd_data['Age'].mean
print(f"   - Welders (n={len(welder_data)}): Age {welder_data['Age'].r

print(f"\n2. MOTOR BALANCE COMPARISON (common symptom data)")
print(f"   Berg Balance Scale (BBS, max=56, cutoff<45):")
print(f"   - PD: {pd_data['BBS'].mean():.1f}+/-{pd_data['BBS'].std():
print(f"   Mini-BESTest (max=28, cutoff<19):")
print(f"   - PD: {pd_data['MiniBEST'].mean():.1f}+/-{pd_data['MiniBEST
print(f"   Falls Efficacy Scale (FES, max=100, cutoff<70):")
print(f"   - PD: {pd_data['FES'].mean():.1f}+/-{pd_data['FES'].std():

print(f"\n3. FALL RISK ANALYSIS")
print(f"   High fall risk (BBS<45):")
print(f"   - PD: {pd_data['high_fall_risk'].sum()}/{len(pd_data)} ({pc
print(f"   - Welders: {welder_data['high_fall_risk'].sum()}/{len(weld

best_group_acc = max(results_group.values(), key=lambda x: x['accurac
best_fall_acc = max(results_fall.values(), key=lambda x: x['accuracy'

print(f"\n4. PREDICTIVE MODEL PERFORMANCE (common symptom features on
print(f"   Group Classification (PD vs Welder):")
print(f"   - Best accuracy: {best_group_acc:.1%} using {best_group}")
print(f"   Fall Risk Classification:")
print(f"   - Best accuracy: {best_fall_acc:.1%} using {best_fall_name

print(f"\n5. SYMPTOM-BASED COMPARISON (common data: BBS, MiniBEST, FES
print(f"   PD - Hoehn & Yahr Stages (1-5) by symptom/balance:")
for stage in sorted(pd_data['Hoehn_Yahr_Stage'].unique()):
    if stage > 0:
        stage_data = pd_data[pd_data['Hoehn_Yahr_Stage'] == stage]
        if len(stage_data) > 0:
            print(f"     - Stage {int(stage)} (n={len(stage_data)}): M
print(f"   Welders - predicted symptom stage (from model trained on PD
if 'Predicted_HY_Stage' in welder_data.columns:
    for s in sorted(welder_data['Predicted_HY_Stage'].dropna().unique
        wsub = welder_data[welder_data['Predicted_HY_Stage'] == s]
        if len(wsub) > 0:
            print(f"     - Predicted stage {int(s)} (n={len(wsub)}): M
print(f"\n   KEY INSIGHT:")
print(f"   - Prediction uses common symptom data only (no welding yea
print(f"   - PD: balance decline by Hoehn & Yahr stage; Welders: mappe
```

```
print(f"   - This comparison reveals distinct motor balance impairmen

print("\n" + "="*70)
```

```
======================================================================
KEY FINDINGS (PD and Welders only; control data not used)
======================================================================

1. SAMPLE CHARACTERISTICS
   - PD patients (n=14): Age 71.7+/-6.2 years, Disease duration 6.5+/-
   - Welders (n=16): Age 41.8+/-6.3 years (symptom-based comparison; w

2. MOTOR BALANCE COMPARISON (common symptom data)
   Berg Balance Scale (BBS, max=56, cutoff<45):
   - PD: 31.9+/-11.6 | Welders: 47.4+/-10.3
   Mini-BESTest (max=28, cutoff<19):
   - PD: 12.3+/-4.5 | Welders: 19.8+/-7.1
   Falls Efficacy Scale (FES, max=100, cutoff<70):
   - PD: 54.1+/-19.5 | Welders: 32.2+/-25.5

3. FALL RISK ANALYSIS
   High fall risk (BBS<45):
   - PD: 11/14 (79%)
   - Welders: 3/16 (19%)

4. PREDICTIVE MODEL PERFORMANCE (common symptom features only)
   Group Classification (PD vs Welder):
   - Best accuracy: 93.3% using Logistic Regression
   Fall Risk Classification:
   - Best accuracy: 96.7% using XGBoost

5. SYMPTOM-BASED COMPARISON (common data: BBS, MiniBEST, FES, ABC)
   PD - Hoehn & Yahr Stages (1-5) by symptom/balance:
     - Stage 1 (n=2): BBS=46.0±2.8, MiniBEST=18.0±5.7, FES=23.0±9.9
     - Stage 2 (n=3): BBS=38.3±11.4, MiniBEST=14.7±4.0, FES=42.0±12.1
     - Stage 3 (n=6): BBS=25.8±7.1, MiniBEST=9.2±2.6, FES=69.8±10.2
     - Stage 5 (n=3): BBS=28.0±14.7, MiniBEST=12.3±3.2, FES=55.7±9.5
   Welders - predicted symptom stage (from model trained on PD H&Y; no
     - Predicted stage 2 (n=16): BBS=47.4±10.3, MiniBEST=19.8±7.1, FES

   KEY INSIGHT:
   - Prediction uses common symptom data only (no welding years); symp
   - PD: balance decline by Hoehn & Yahr stage; Welders: mapped to sam
   - This comparison reveals distinct motor balance impairment pattern

======================================================================
```

10. End result — text summary

Run the cell below to print the main results as plain text (for reporting or copying).

```
# ======================================================================
# END RESULT — PRINT ALL KEY RESULTS AS TEXT
# ======================================================================
def _p(txt=""):
    print(txt)

_p("=" * 80)
_p("END RESULT — MOTOR BALANCE ANALYSIS (PD vs WELDERS)")
_p("=" * 80)
_p()

# 1. Sample
pd_n = len(analysis_data[analysis_data['Group'] == 'PD'])
wd_n = len(analysis_data[analysis_data['Group'] == 'Welder'])
_p("1. SAMPLE")
_p(f"   PD:      n = {pd_n}")
_p(f"   Welders: n = {wd_n}")
_p("   (Control data not used.)")
_p()

# 2. Clinical comparison (means ± SD)
_p("2. CLINICAL SCORES (mean ± SD)")
for grp in ['PD', 'Welder']:
    d = analysis_data[analysis_data['Group'] == grp]
    _p(f"   {grp}:  BBS = {d['BBS'].mean():.1f} ± {d['BBS'].std():.1f
_p()

# 3. Fall risk
_p("3. HIGH FALL RISK (BBS < 45)")
for grp in ['PD', 'Welder']:
    d = analysis_data[analysis_data['Group'] == grp]
    n_high = d['high_fall_risk'].sum()
    pct = 100 * n_high / len(d) if len(d) > 0 else 0
    _p(f"   {grp}: {int(n_high)} / {len(d)} ({pct:.0f}%)")
_p()

# 4. Model results
_p("4. MODEL PERFORMANCE (LOOCV)")
```

```
if 'results_severity' in globals() and results_severity:
    b = max(results_severity, key=lambda k: results_severity[k]['accu
    _p(f"   Motor Balance Severity (1-5):  Best = {b}  |  Accuracy = 
if 'results_group' in globals() and results_group:
    b = max(results_group, key=lambda k: results_group[k]['accuracy']
    _p(f"   Group (PD vs Welder):          Best = {b}  |  Accuracy = 
if 'results_fall' in globals() and results_fall:
    b = max(results_fall, key=lambda k: results_fall[k]['accuracy'])
    _p(f"   Fall risk (high vs low):       Best = {b}  |  Accuracy = 
if 'results_hy' in globals() and results_hy:
    b = max(results_hy, key=lambda k: (results_hy[k]['accuracy'], resu
    _p(f"   H&Y stage (PD → Welders):      Best = {b}  |  Accuracy = 
_p()

# 5. Welders predicted stage
_p("5. WELDERS — PREDICTED SYMPTOM STAGE (H&Y-like 1–5)")
wd = analysis_data[analysis_data['Group'] == 'Welder']
if 'Predicted_HY_Stage' in wd.columns and wd['Predicted_HY_Stage'].no
    counts = wd['Predicted_HY_Stage'].value_counts().sort_index()
    for stage, c in counts.items():
        _p(f"   Stage {int(stage)}: {int(c)} welder(s)")
    _p("   (Model trained on PD only; common clinical features.)")
else:
    _p("   (Run Section 7.0 to get predicted stages.)")
_p()

# 6. Takeaway
_p("6. TAKEAWAY")
_p("   • Analysis uses PD + Welders only; prediction uses common symp
_p("   • Best model for H&Y-stage prediction on PD is applied to weld
_p("   • All metrics from LOOCV on training data; welder predictions 
_p()
_p("=" * 80)
_p("END OF RESULTS")
_p("=" * 80)
```

```
================================================================================
END RESULT — MOTOR BALANCE ANALYSIS (PD vs WELDERS)
================================================================================

1. SAMPLE
   PD:      n = 14
   Welders: n = 16
   (Control data not used.)

2. CLINICAL SCORES (mean ± SD)
```

```
   PD:  BBS = 31.9 ± 11.6  |  MiniBEST = 12.3 ± 4.5  |  FES = 54.1 ± 1
   Welder:  BBS = 47.4 ± 10.3  |  MiniBEST = 19.8 ± 7.1  |  FES = 32.2

3. HIGH FALL RISK (BBS < 45)
   PD: 11 / 14 (79%)
   Welder: 3 / 16 (19%)

4. MODEL PERFORMANCE (LOOCV)
   Motor Balance Severity (1-5):  Best = Random Forest  |  Accuracy = 
   Group (PD vs Welder):          Best = Logistic Regression  |  Accur
   Fall risk (high vs low):       Best = XGBoost  |  Accuracy = 0.967
   H&Y stage (PD → Welders):      Best = M4_FES  |  Accuracy = 0.500

5. WELDERS — PREDICTED SYMPTOM STAGE (H&Y-like 1–5)
   Stage 2: 16 welder(s)
   (Model trained on PD only; common clinical features.)

6. TAKEAWAY
   • Analysis uses PD + Welders only; prediction uses common symptom d
   • Best model for H&Y-stage prediction on PD is applied to welders t
   • All metrics from LOOCV on training data; welder predictions are o

======================================================================
END OF RESULTS
======================================================================
```